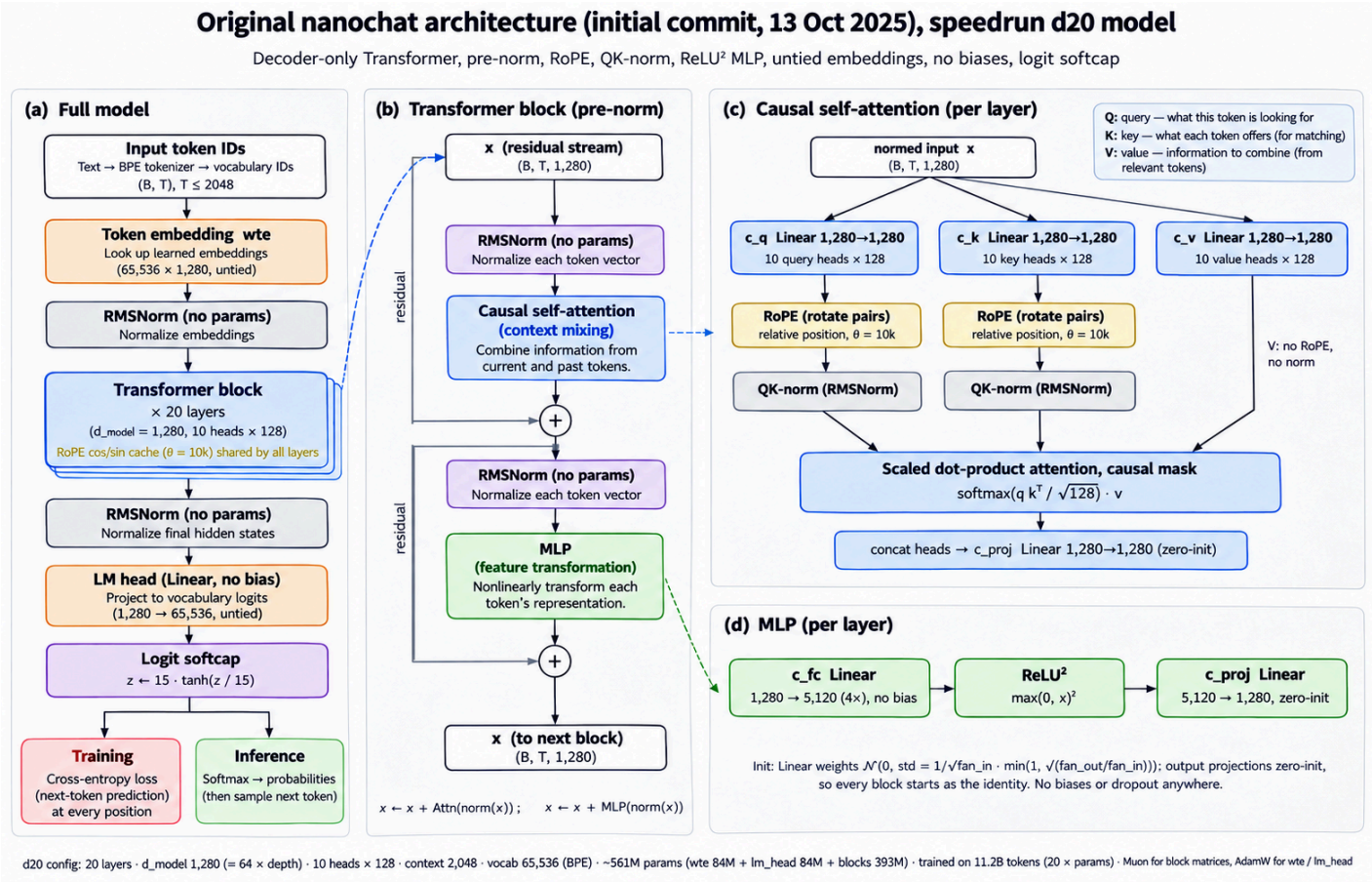
A3: Pre-training Nanochat

By: Riyad Valiyev (Student Number: 1009069874), Boquan Zhao (Student Number: 1010419343),
Tanayjyot Singh Chawla (Student Number: 1010411109), Carmen Chau (Student Number: 1009043376)

Part 1: Architecture Review

1.1: Original Nanochat Architecture Diagram

Diagram was made based on the initial Nanochat commit: see here: [October 13th, 2025 commit].



Architectural Diagram interpretation: We observe that Nanochat is a GPT-style decoder-only Transformer (generates text autoregressively, predicting each token using only preceding tokens). This distinguishes NanoChat from BERT (introduced in Lecture 4) and ModernBERT, which are encoder-only Transformers using bidirectional self-attention to learn contextual representations. Unlike encoder–decoder Transformers, nanochat has neither a separate encoder nor cross-attention.

Although GPT-2 is also a decoder-only Transformer, nanochat introduces several architectural and training optimizations:

- **Attention:** Projects token representations into queries (Q), keys (K), and values (V). Q–K comparisons determine attention weights used to aggregate V. Full causal self-attention is implemented using PyTorch SDPA, with KV caching for efficient inference.
- **Positional encoding:** Uses rotary positional embeddings (RoPE) on Q and K instead of GPT-2's learned absolute positional embeddings.
- **Normalization:** Replaces GPT-2's LayerNorm with parameter-free RMSNorm, and additionally normalizes Q and K to improve attention stability.
- **MLP and simplifications:** Uses ReLU² instead of GELU, removes biases and dropout, unties input/output embeddings, and zero-initializes block output projections to preserve the identity mapping initially.
- **Output:** Applies logit softcapping before cross-entropy loss to limit extreme prediction score

1.2: Possible Architecture Changes

Drawing on recent LLM architecture research, we propose three modifications to nanochat targeting distinct components of the model: attention refinement **(Differential Attention V2)** to suppress irrelevant contextual information, effective depth and parameter sharing (**Looped Transformer with per-loop embeddings**) to enable iterative computation, and local token mixing (**Canon-AC layer**s) to capture short-range dependencies throughout the network.

Proposed Change	**Description + Motivation**	**Technical detail of the change**	**Potential impact on model performance**

Differential Attention V2 (Ye et al., 2026)
 	
For reference: See Differential Attention V1 paper (Ye et al., 2025).
	Differential Attention V2 (Diff V2) is a newer version of the Differential Attention architecture (Diff V1), introduced by authors Ye et al (2025).
	
	Diff V1 aimed to **reduce attention noise**, created when conventional softmax functions assign non-negligible weights to tokens with very little relevance to the query. However, it introduced additional inference overhead and potential gradient instability due to its per-head RMSNorm.
	
	Although Nanochat incorporates efficient attention mechanisms such as FlashAttention and sliding-window attention, it
		Without Diff V2: By default, Nanochat uses conventional multi-head scaled dot-product attention, where each query head produces a softmax attention distribution to weight value vectors.
		
		With Diff V2: Diff V2 replaces this with differential attention, where two attention outputs are computed and subtracted using a learnable scaling factor, λ:
		
		$O_{t,i} = \mathrm{Attn}(q_{t,2i}, K_i, V_i) - \lambda_{t,i}\,\mathrm{Attn}(q_{t,2i+1}, K_i, V_i)$. Here, $O_{t,i}$ denotes the differential attention output vector for token t and head pair i.
		
		When Diff V2 is enabled in Nanochat (via flag--diff-v2), we modify the `CausalSelfAttention` class in `nanochat/gpt.py` to double the number of query heads and initialize a learnable projection to compute the subtraction coefficient:
		
		`self.n_q_head = self.n_head * (2 if self.diff_v2 else 1)`
		`self.head_dim = self.n_embd // self.n_head`
			Positive: Model quality: Ye et al. (2026) report a 0.02-0.03 reduction in language-modeling loss compared with a conventional softmax-based Transformer after 1T training tokens. Although Nanochat would be trained on fewer tokens, Diff V2's ability to suppress irrelevant attention scores could still improve Nanochat's language-modeling performance by potentially lowering validation BPB and improving CORE.
			
			Positive (Training stability and inference): Diff V2 demonstrated fewer loss/gradient spikes and smaller activation outliers during large-scale pretraining (Ye et al., 2026). This could help Nanochat train more consistently and reduce the risk of unstable convergence. Furthermore, Diff V2's

	retains conventional softmax-based attention. This motivates us to investigate Diff V2 as a potential architectural improvement for Nanochat that mitigates attention noise while balancing training stability and computational efficiency.	`self.c_q = Linear(self.n_embd, self.n_q_head * self.head_dim, bias=False)`
		`... (omitting writing self.c_k, self.c_v for previty)`
		`self.lambda_proj = (`
		`  Linear(self.n_embd, self.n_head, bias=False)`
		`  if self.diff_v2 else None`
		
		Using **Grouped Query Attention (GQA)**, every two query heads share the same keys and values but independently compute attention outputs. Diff V2 then uses a sigmoid projection to calculate a separate scaling factor, $\lambda_{□,i}$, for each token and differential head, before subtracting the paired outputs:
		
		`if self.diff_v2:`
		`  lam = torch.sigmoid(self.lambda_proj(x)).to(y).unsqueeze(-1)`
		`  y = y[:, :, 0::2] - lam * y[:, :, 1::2]`
		
		`y = y.contiguous().view(B, T, -1)`
		`y = self.c_proj(y)`
		
		The resulting differential attention head outputs are then concatenated and passed through Nanochat's existing output projection, preserving the standard multi-head attention structure. This preserves the original output dimension of `n_embd` (eg: 768 for depth=12).
		
		Unlike Diff V1, Diff V2 also omits the additional per-head RMSNorm after differential attention to address potential training instability. Its GQA-based design leverages Nanochat's existing FlashAttention support without requiring custom attention kernels (Ye et al., 2026).
			shared KV heads enable reuse of Nanochat's existing FlashAttention backend and KV-cache dimensions, potentially limiting inference overhead and increasing architecture compatibility relative to Diff V1.
			
			Negative (Computational cost): Ye et al. (2026) acknowledged that Diff V2 introduces additional query projections and parameters compared with conventional Transformers, potentially increasing training FLOPs and wall-clock time. Since their reported model quality improvements come from much larger models, the benefits may not transfer to Nanochat/Picochat. This raises a trade-off that we need to investigate between model quality and wall-clock efficiency, which Karpathy (2026) emphasizes when evaluating Nanochat modifications.

Looped Transformer with per-loop	Looped Transformer with per-loop embeddings (Looped-l2-embed) increases a	**Without Looped-l2-embed:** By default, Nanochat executes each Transformer block once, with separate attention and	**Positive (Reasoning performance):** Saunshi et al. (2025) found that a 12-layer Transformer looped twice

embedding (Looped-l2-embed) (Geiping et al., 2025; Saunshi et al., 2025; Zhu et al., 2025)	model's effective depth by applying the same core Transformer blocks **twice with shared weights**. A learned embedding vector is added to every token's hidden state at the start of each core pass to explicitly identify the pass. Inspired by the timestep embeddings in Universal Transformers (Dehghani et al., 2019) and loop-specific adaptations in Relaxed Recursive Transformers (Bae et al., 2025), we hypothesize that these lightweight embeddings encourage complementary processing across passes with minimal additional parameters. This architecture offers an alternative to Nanochat's default scaling recipe: instead of increasing depth and width together, looping increases effective depth while reusing existing weights. Base looped-l2 aims to improve reasoning through additional sequential computation without proportionally increasing model	MLP weights for each block. (Eg: A d = 12 model therefore performs 12 block executions.) **With Looped-l2-embed enabled:** In the literature, looped Transformers repeatedly apply shared Transformer blocks to updated hidden representations. The attention and MLP computations run again, but their weight matrices are reused across passes. Geiping et al. (2025) organize this computation into a **prelude**, a **shared recurrent core**, and a **coda**. *Implementing looped-l2 logic:* We adopt Geiping et al.'s prelude-core-coda structure with a **fixed two-pass schedule**. The effective depth is: $D_{\text{effective}} = P + L(d - P - C) + C$ where d = depth, aka number of unique blocks, p = prelude size, c = coda sizes, and L = number of recurrent passes, specified independently using --n-loops. Eg: For our d12 model, we set $P = C = 1$ and $L = 2$: $D_{effective} = 1 + 2(12 - 1 - 1) + 1 = 22$ Thus, 12 unique blocks perform 22 executions, with 10 shared core blocks executed twice. In `nanochat/gpt.py`, we implement the above equation like so: `return ( list(range(self.n_prelude)) + core * n_loops + list(range(self.n_layer - self.n_coda, self.n_layer)) )`	outperformed a conventional 24-layer Transformer on math word problems (34.3 vs. 29.3) and reasoning primitives (51.2 vs. 47.5), despite having fewer parameters and worse validation perplexity. Similarly, Geiping et al. (2025) and Zhu et al. (2025) demonstrated improvements in reasoning through recurrent computation. These findings suggest that introducing looping into Nanochat could improve reasoning-related performance on CORE benchmarks by allowing the model to iteratively refine hidden representations without adding new Transformer blocks. **Positive (Parameter efficiency and pass-specific learning):** Zhu et al. (2025) found that looping improves knowledge manipulation without proportionally increasing knowledge storage capacity. Furthermore, Bae et al. (2025) demonstrated that loop-specific adaptations can improve the effectiveness of shared Transformer weights. Our per-loop embeddings similarly aim to encourage different processing behaviors across passes. This could improve Nanochat's ability to

	size. Saunshi et al. (2025) show that a k-layer Transformer looped L times can nearly match a kL -layer model on several reasoning tasks. Geiping et al. (2025) similarly demonstrate reasoning gains from recurrent latent computation, while Zhu et al. (2025) find that looping improves knowledge manipulation without increasing knowledge storage capacity in controlled experiments.
	
	However, while weight sharing limits parameter growth, additional passes increase computational cost and runtime. We therefore investigate whether improvements in validation BPB and reasoning performance justify the additional computation relative to the non-looped baseline.
		The forward pass reuses `self.transformer.h[b]`, executing the same attention and MLP weights on updated hidden states.
		
		Implementing Per-loop embeddings: Beyond Nanochat's standard residual computation, we add a learned embedding $e_\ell \in \mathbb{R}^{d_{\text{model}}}$ at the start of each shared core pass so that the reused blocks can distinguish which iteration they are processing. Conceptually, this can be written as: $H^{(\ell)} = F(H^{(\ell-1)} + e_\ell)$ where F denotes the shared Transformer core and $H^{(\ell-1)}$ denotes all tokens' hidden states entering pass ℓ.
		
		Alongside Nanochat's residual computation, these additional embeddings can be allocated in Nanochat in `nanochat/gpt.py` like so:
		
		`x = self.resid_lambdas[s] * x + self.x0_lambdas[s] * x0`
		
		`if self.loop_embeds is not None and i in plan["pass_starts"]:`
		`  x = x + self.loop_embeds[plan["pass_starts"][i]].to(x.dtype)`
		
		Each recurrent pass introduces one learned embedding vector of dimension d_{model}. Thus, the total number of additional parameters relative to plain looping is: $N_{embed} = L \cdot d_{model}$ where L denotes the number of recurrent passes and d_{model} is the hidden embedding dimension. For our d12 model, with $L = 2$ passes and $d_{model} = 768$.
		$N_{embed} = 2 \times 768 = 1,536$
		These embeddings are zero-initialized, so the model initially behaves identically to plain looping with the same weights,
			leverage its existing parameters for more complex reasoning tasks.
			
			Negative (Computational cost and model quality): Although looping limits parameter growth, repeated attention and MLP computations increase training FLOPs, memory usage, and wall-clock time. Saunshi et al. (2025) also found that looped models can exhibit worse perplexity than non-looped models with equivalent computational cost. Since much of the supporting research uses substantially larger models, these benefits may not transfer directly to Nanochat. We therefore investigate whether potential improvements in validation BPB and CORE justify the increased computational cost, particularly against a compute-matched baseline.

		while allowing pass-specific adjustments to be learned during training.	
3. Canon layers *(Allen-Zhu, NeurIPS 2025)*	Transformer blocks primarily rely on attention to exchange information between tokens, even for simple local dependencies. Although Nanochat's *smear* mixes each token's embedding with its predecessor, this occurs only at the input. Canon layers extend learned local mixing to intermediate representations throughout the network, potentially reducing reliance on attention for short-range patterns.	Canon layers introduce a **causal depthwise 1-D convolution** with residual connection: $h'_t = h_t + \sum_{i=0}^{3} w_i \odot h_{t-i}$, allowing each token to incorporate its previous three positions through learned per-channel weights. The paper explores placements before attention (A), within attention projections (B), before the MLP (C), and inside the MLP (D). For potential integration into Nanochat, we propose **Canon-AC**, combining Canon-A and Canon-C rather than using either individually. The paper's architecture changes show **cumulative performance improvements when combining Canon placements**, motivating local mixing before both attention and MLP sublayers instead of only one. This allows both sublayers to process representations enriched with neighbouring-token information while avoiding changes to their internal operations. In Nanochat, we would insert a causal depthwise convolution after normalization and before each attention and MLP sublayer, preserving Flash Attention 3 and the existing MLP architecture. Each site adds only (4d) parameters and requires caching preceding hidden states during inference.	**Positive:** The paper reports **2–4× greater reasoning depth** and approximately **30% greater reasoning breadth** on synthetic tasks (primarily with Canon-ABCD). Its ablations also show that Canon-AC performs strongly, suggesting that local mixing can improve learning without modifying attention itself. For Nanochat, this could improve contextual representations and reduce validation BPB. **Negative:** Real-world pretraining improvements at 1.3B parameters remain difficult to distinguish from seed variability. Despite minimal parameter overhead, Canon-AC adds approximately **5.8% training runtime overhead** and **7% generation overhead** in the paper's H100 experiments. At Nanochat's smaller scale, any quality gains may not justify the additional computation.

Part 2: Ablations on a Picochat

In this part we train a small nanochat model, the “picochat”, to test the two changes chosen in Part 1. Each change is trained alone against an unchanged baseline, and we compare them twice: at an equal number of training tokens and at equal training compute. The main findings:

- **Both changes help at equal tokens.** On 1.32B tokens, Diff V2 lowers validation bits per byte (val bpb) by 0.0066 and Looped-l2-embed by 0.0102 (seed 42). Those gaps are six and nine times the 0.0011 bpb spread between baseline seeds, and the looped gain holds across three seeds (0.0101).
- **Neither change beats simply training the baseline for longer.** Given Looped-l2-embed's compute, the baseline reaches 0.8280 bpb, 0.0098 better than looping. Diff V2 ends 0.0001 bpb behind a baseline interpolated to its compute, so its gain is about what its extra 19% compute would buy anyway.
- **CORE is too noisy at this size to separate the changes.** Looped-l2-embed scores highest (0.1806 at seed 42), but the baseline's three seeds span 0.017 and two identical runs differ by 0.005, so CORE gaps under about 0.02 cannot be trusted.
- **The per-pass embedding adds a little.** Looped-l2 without it ties on bpb but scores lower on CORE on all three seeds (mean gap 0.0074), so we kept the embedding.
- **All nine runs cost about $42.60** on one H100 each (10.8 H100-hours).

2.1 Picochat baseline configuration

We run our ablations on a smaller nanochat model, the picochat. Table 2.1 lists its configuration, as each run logs it at start-up.

Property	Value	What it means
Depth (n_layer)	12	Number of transformer layers
Model width (n_embd)	768	Features in each token's vector; set automatically to depth × 64, nanochat's default aspect ratio
Query heads (n_head)	6	Number of attention heads
Key/value heads (n_kv_head)	6	Equal to the query heads, so each query head has its own keys and values
Head dimension (head_dim)	128	Size of each query, key and value head

Context length (sequence_len)	2,048 tokens	Longest sequence the model sees
Attention pattern (window_pattern)	SSSL	Three sliding-window layers, then one full-context layer, repeated
Vocabulary	32,768 tokens	The team tokenizer (Section 2.2)
Training steps	2,520	Fixed for every run at equal tokens (Section 2.4)
Batch	524,288 tokens per step	32 sequences of 2,048 tokens per micro-batch, with 8 gradient-accumulation steps
Training tokens	1.32B	2,520 × 524,288
Parameters	286.3M in total	84.9M in the transformer blocks, 151.0M in value embeddings, and 25.2M each in the input embedding and output head
Scaling parameters	110.1M	The count nanochat uses to set the token budget

Table 2.1. Picochat baseline configuration.

Why depth 12. Our main change from nanochat's defaults is cutting the depth from 20 layers to 12, following Karpathy's recommendation of depth-12 models for quick ablation experiments (Karpathy, 2025–2026). Twelve layers is also a standard baseline depth in established language models such as BERT-Base (Devlin et al., 2018). Through nanochat's depth-based scaling rules, this lowers the width from 1,280 to 768 and the number of attention heads from 10 to 6, which cuts the total parameters from about 896.5M to 286.3M, a 68% reduction. This size suits our experiments for two more reasons:

- **It is cheap enough to repeat.** A baseline run costs about $3.45 on one H100, which let us run three seeds for the baseline and Looped-l2-embed and two compute-matched controls.
- **It is big enough to show an effect.** At d12 the gaps between changes (0.006–0.010 bpb) are many times the noise between seeds (0.0011 bpb), and CORE is clearly above chance (about 0.15).

We keep nanochat's default head dimension, context length and attention pattern. Following nanochat's default of 12 training tokens per scaling parameter, we fix the training budget at 2,520 steps, or 1.32B tokens, the same ratio we use for the d20 model in Part 5. A fixed token budget gives every architecture the same data to learn from.

2.2 Dataset and tokenizer

- **Tokenizer.** We use nanochat's byte-level Byte Pair Encoding (BPE) tokenizer, trained with scripts/tok_train.py on text from the ClimbMix training split. We keep the default 32,768-token vocabulary and the 10,000-character cap per document, but halve

the training text from 2 billion to 1 billion characters to limit preprocessing; the limit was reached in the fifth shard. Every run in this report, including the d20 runs in Part 5, uses this tokenizer.

- **Training data.** We pretrain on 32 ClimbMix training shards (about 1.43B tokens) and measure validation bpb on a separate held-out shard. The 1.32B-token budget reads each training token once. We chose 32 shards to give the main runs enough fresh data while limiting download and storage.

2.3 Ablations selected

From the changes proposed in Part 1, we test Differential Attention V2 (Diff V2) and the looped Transformer with a per-pass embedding (Looped-l2-embed); Section 1.4 gives the full reasoning. In short, they act on different parts of the model, both are new to nanochat, and both make a clear, testable prediction. The team's exploratory runs also favoured them: both showed promising gains in val bpb and CORE, while Canon layers gave only marginal ones.

2.4 Experimental protocol

We train each change separately against the picochat baseline, each run on one NVIDIA H100 80GB GPU on Modal with Flash Attention 3 and bf16. Both changes spend more compute per token than the baseline (Part 1), so an equal-token comparison could credit them with gains that really come from the extra compute rather than from the architecture. We therefore compare at equal tokens and at equal compute. Table 2.2 lists the nine runs.

Run	What changes	Steps	Tokens	FLOPs per token	Total compute vs. baseline	Seeds	W&B runs
Baseline	Nothing: nanochat as it is	2,520	1.32B	7.60e8	×1.00	42, 1, 2	2msa7208, 24wheiew, cgwmsfqi
Diff V2	Paired query heads with a learned subtraction (weights 84.9M → 92.1M, +8%)	2,520	1.32B	9.02e8 (+19%)	×1.19	42	u4asolds
Looped-l2-embed	1 prelude block, 10 shared blocks run twice and 1 coda block (22 executed layers), with a learned vector added at the start of each pass	2,520	1.32B	12.74e8 (+68%)	×1.68	42, 1, 2	cqcpx9lb, w6cud06w, efl63fy0
Baseline-long	The baseline trained for longer, to Looped-l2-embed's total compute	4,224	2.21B	7.60e8	×1.68	42, run twice	brihic8u, ys0w0wws

Table 2.2. The nine Part 2 runs, all d12 on one H100. Total compute is FLOPs per token × tokens, relative to the baseline.

Controls. Every run shares the same code, tokenizer, data and data order, learning-rate schedule and hardware:

- **One change at a time.** Each run selects its change with an explicit --experiment flag, which rejects the other change's flags, so no run combines changes by accident. With the change switched off, the code path is the baseline's.
- **A pinned budget.** The step count and batch size are fixed to the baseline's values. Otherwise nanochat sizes the run from the parameter count and would give Diff V2, which adds parameters, a 6% longer schedule.
- **A compute-matched control.** Baseline-long trains the baseline for 4,224 steps (2.21B tokens) on the same 32 shards, which gives it Looped-l2-embed's total compute (4,224 × 7.60e8 = 2,520 × 12.74e8 FLOPs per token, within 0.05%). For Diff V2 we estimate the baseline at equal compute by interpolating between the baseline and baseline-long in log compute.
- **Measured noise.** The main runs use seed 42. The baseline and Looped-l2-embed also ran at seeds 1 and 2, which change only the initial weights. A launcher restart ran baseline-long twice with identical settings, which measures pure run-to-run noise.
- **Metrics.** Validation bpb on 42M held-out tokens every 250 steps, and the CORE score at the end: the mean centered accuracy over nanochat's 22 in-context-learning tasks (up to 500 examples each), where 0 is random guessing and 1 is perfect.

The code is in the team repository, with the implementation in PR #28 and the training logs in PR #30: the model in gpt.py, the experiment flags in experiments.py, the Modal launcher in train_d12.py and tests under tests/ablations. Every training curve is in Weights & Biases (Section 2.10).

2.5 Results at equal tokens

Run	Transformer weights	FLOPs per token	Val bpb	Δ bpb	CORE	Δ CORE	3-seed val bpb	3-seed CORE
Baseline	84.9M	7.60e8	0.8480	–	0.1608	–	0.8476 ± 0.0006	0.1537 ± 0.0088
Diff V2	92.1M (+8%)	9.02e8 (+19%)	0.8415	−0.0066	0.1727	+0.0120	–	–
Looped-l2-embed	84.9M	12.74e8 (+68%)	**0.8378**	**−0.0102**	**0.1806**	**+0.0198**	0.8375 ± 0.0003	0.1763 ± 0.0054
Baseline-long*	84.9M	7.60e8 (×1.68 tokens)	0.8280	−0.0201	0.1666	+0.0059	–	–

*Table 2.3. Final results. For the baseline model, numbers for the column Transformer weights to CORE are metrics are from seed-42; Δ is against the seed-42 baseline, and negative Δ bpb and positive Δ CORE are improvements. The 3-seed columns are mean ± standard deviation over seeds 42, 1 and 2; against the 3-seed baseline, Looped-l2-embed's gain is −0.0101 bpb and +0.0226 CORE. *Mean of two identical runs on 2.21B tokens.*

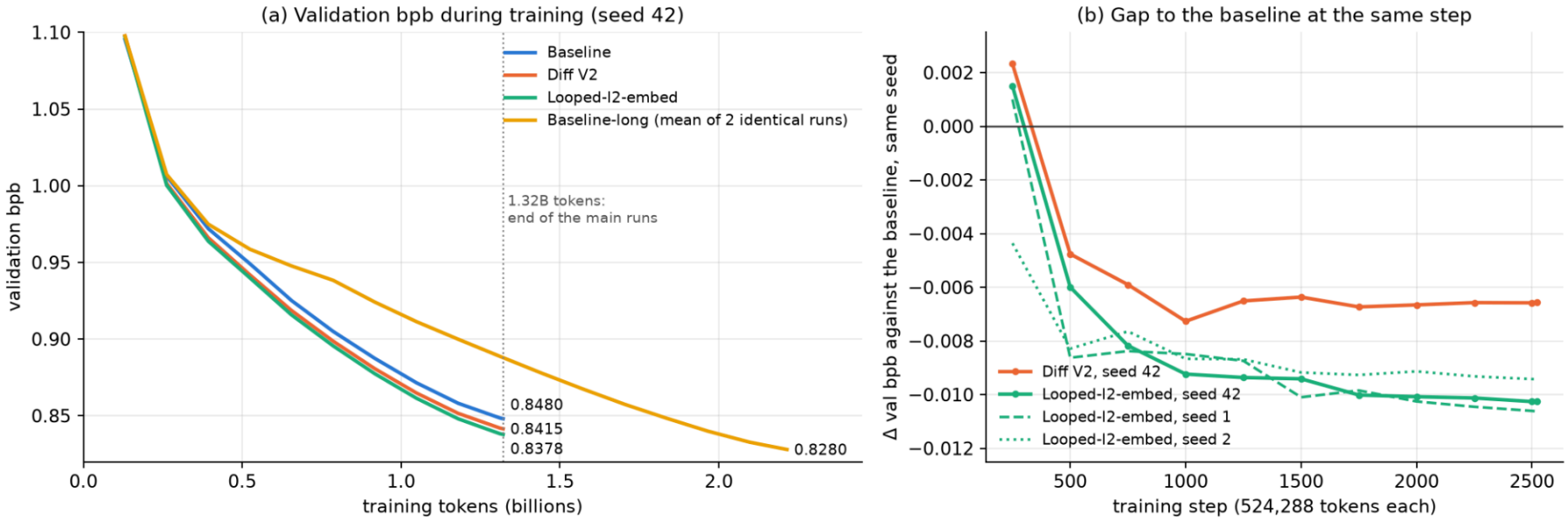


Figure 2.1. (a) Validation bpb during training for the seed-42 runs; baseline-long continues to 2.21B tokens on its own, longer schedule. (b) Each change's gap to the baseline at the same step and seed. For Looped-l2-embed, the three seeds are each compared with the baseline of the same seed.

Run	***Step 250***	***500***	***1,000***	***1,500***	***2,000***	***2,520 (end)***
Baseline	*1.0954*	*1.0062*	*0.9491*	*0.9049*	*0.8715*	*0.8480*
Diff V2	*1.0977*	*1.0015*	*0.9418*	*0.8986*	*0.8648*	*0.8415*
Looped-l2-embed	*1.0969*	*1.0003*	*0.9399*	*0.8955*	*0.8614*	*0.8378*

Table 2.4. Validation bpb during training (seed 42). Every run starts at 3.17 at step 0.

Analysis:

Verdict: **Both changes help, and the gains are far above the noise.**

Looped-l2-embed gives the larger gain (−0.0102 bpb, +0.0198 CORE at seed 42). Its lead grows through training, from 0.006 at step 500 to 0.010 at the end, and the other two seeds end 0.0106 and 0.0094 ahead of their own baselines. The price is compute: it executes 22 layers instead of 12, so it spends 68% more FLOPs per token.

Diff V2 starts slightly behind (1.0977 against 1.0954 at step 250), moves ahead by step 500, and from step 1,250 holds a steady lead of about 0.0065 to finish 0.0066 ahead. It also improves CORE by 0.012. It spends 19% more FLOPs per token and has 8% more weights, which Section 2.6 accounts for.

For scale: the baseline's three seeds end within 0.0011 bpb of each other, and the two identical baseline-long runs within 0.0001. Both changes' bpb gains are many times larger.

Connection to the architectural research in Part 1:

Diff V2's improvement is consistent with its learned subtraction of paired attention outputs, which aims to suppress irrelevant contextual information within nanochat's existing attention layers. Looped-l2-embed achieves a larger BPB reduction by executing 22 effective layers instead of 12, allowing representations to undergo additional rounds of processing through shared attention and MLP blocks. Unlike simply increasing model depth, this reuses existing weights, separating computational depth from parameter count. However, both improvements come with additional FLOPs per token, motivating the compute-matched analysis.

2.6 Results at equal compute

An equal-token comparison favours any change that spends more compute per token. The fairer question is whether the extra compute would have done more good if the baseline had simply trained for longer. Table 2.5 answers it for each change, and Figure 2.2 plots every run against its cumulative training compute.

Run	Compute vs. baseline	Val bpb	Baseline at the same compute	Δ bpb	CORE	Baseline CORE at the same compute	Δ CORE
Looped-l2-embed	×1.68	0.8378	0.8280 (baseline-long, measured)	+0.0098	0.1806	0.1666 (measured)	+0.0139
Diff V2	×1.19	0.8415	≈0.8414 (interpolated)	≈+0.0001	0.1727	≈0.1627 (interpolated)	≈+0.010

Table 2.5. Each change against the baseline trained to the same total compute (seed 42). Positive Δ bpb means the change is worse than training the baseline for longer. The interpolation is linear in log compute between the baseline (×1.00) and baseline-long (×1.68).

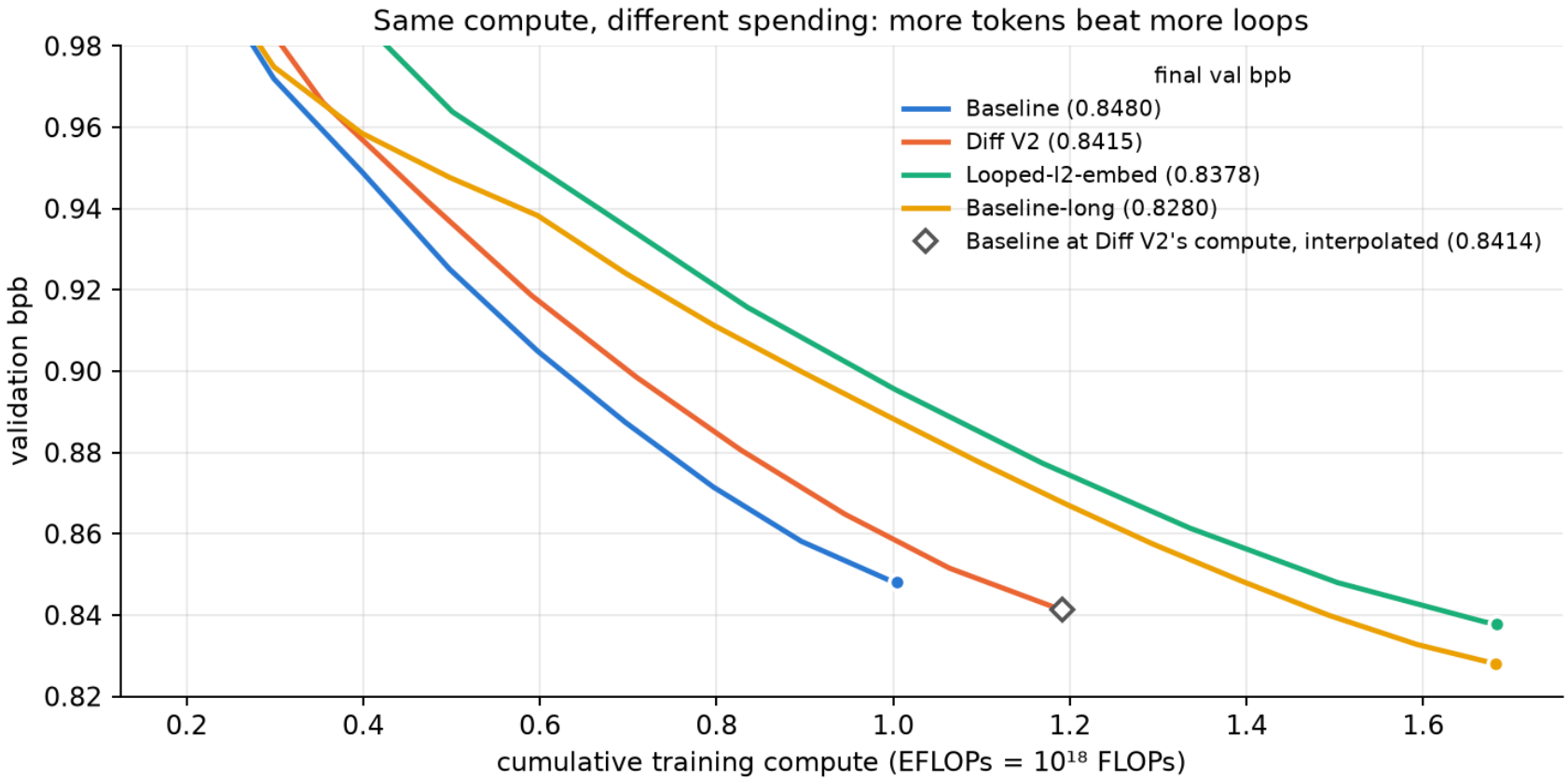


Figure 2.2. Validation bpb against cumulative training compute (seed 42; baseline-long is the mean of its two identical runs). At Looped-l2-embed's compute, baseline-long reaches 0.8280 and Looped-l2-embed 0.8378. The diamond is the interpolated baseline at Diff V2's compute.

Analysis:

Spending Looped-l2-embed's extra compute on more tokens beats spending it on loops, by 0.0098 bpb at seed 42 (0.0095 on the 3-seed mean). That is about as large as looping's whole gain at equal tokens, so its bpb advantage in Section 2.5 comes from its extra compute, not from sharing weights. Its CORE lead over baseline-long (+0.014 at seed 42, +0.010 on the 3-seed mean) is within the CORE noise described in Section 2.10.

Diff V2's bpb gain is almost exactly what its extra 19% compute would buy (0.8415 against an interpolated 0.8414). Its CORE gain of about +0.010 survives the correction, but it rests on one seed and sits within the same noise.

Time is a second cost. Diff V2's training loop took 27% longer than the baseline's, more than its 19% FLOP overhead, because its extra element-wise work lowers MFU (39.0% against 41.6%). Looping kept the baseline's MFU, so its time grew in line with its FLOPs (×1.68).

Connection to the architectural research in Part 1: Despite Looped-l2-embed's additional processing depth, Baseline-long achieves better validation BPB at equivalent compute. This suggests that, at d12, spending additional computation on more optimization steps is more effective for general language modeling than repeatedly applying shared Transformer blocks.

Similarly, Diff V2's BPB approaches the interpolated compute-matched baseline, suggesting that its proposed attention-noise suppression provides little measurable benefit beyond additional compute at this scale. However, these findings do not rule out advantages on specific downstream tasks, which we investigate through CORE.

2.7 Per-task CORE

CORE averages 22 tasks, so a change could help some tasks and hurt others while barely moving the average. Table 2.6 gives every task's centered score, and Figure 2.3 shows each change's difference from the baseline at seed 42.

CORE task	Baseline	Diff V2	Looped-l2-embed	Baseline-long*	Baseline, 3 seeds	Looped-l2-embed, 3 seeds
Commonsense and language inference						
HellaSwag (0-shot)	0.1760	0.1707	0.1760	0.2053	0.1565	0.1911
HellaSwag (10-shot)	0.1733	0.1760	0.1733	0.1853	0.1573	0.1884
COPA	0.1200	0.1200	0.2600	0.1800	0.1200	0.2000
CommonsenseQA	0.2175	0.1525	0.2050	0.1250	0.1750	0.1642
Winograd	0.1429	0.2161	0.1795	0.1941	0.1746	0.1648
Winogrande	0.1000	0.1000	0.0640	0.1120	0.0600	0.0520
PIQA	0.3840	0.3960	0.4040	0.4140	0.3747	0.3827

Knowledge and question answering						
BIG-bench QA Wikidata	0.2400	0.2620	0.3180	0.3090	0.2733	0.3207
ARC-Easy	0.4213	0.4480	0.4560	0.4533	0.4284	0.4738
ARC-Challenge	0.0613	0.0720	0.0800	0.0640	0.0471	0.0693
OpenBookQA	0.1067	0.0933	0.0907	0.1040	0.0969	0.0880
Jeopardy	0.0020	0.0140	0.0120	0.0080	0.0060	0.0140
LAMBADA (OpenAI)	0.3240	0.3260	0.3260	0.3390	0.3087	0.3260
SQuAD	0.1820	0.2160	0.2280	0.1800	0.1747	0.2267
CoQA	0.1880	0.1880	0.2220	0.2040	0.1687	0.2100
BoolQ	−0.2211	−0.0895	−0.1895	−0.3000	−0.2263	−0.1439
Reasoning and symbolic tasks						
BIG-bench CS Algorithms	0.4500	0.4120	0.4560	0.4420	0.4493	0.4580
BIG-bench Operators	0.1238	0.1571	0.1000	0.1142	0.1175	0.1159
BIG-bench Repeat Copy Logic	0.0000	0.0000	0.0000	0.0000	0.0000	0.0208
BIG-bench Dyck Languages	0.0640	0.0940	0.1620	0.0880	0.0787	0.1287
AGI-Eval LSAT-AR	0.0707	0.0761	0.1141	0.0489	0.0344	0.0652
Other						
BIG-bench Language Identification	0.2101	0.1991	0.1353	0.1958	0.2064	0.1624
CORE	**0.1608**	**0.1727**	**0.1806**	**0.1666**	**0.1537**	**0.1763**

*Table 2.6. CORE by task in centered scores (0 = random guessing, 1 = perfect). The first four columns are seed 42 on 1.32B tokens, except baseline-long on 2.21B. *Mean of two identical runs, task by task. The last two columns average seeds 42, 1 and 2.*

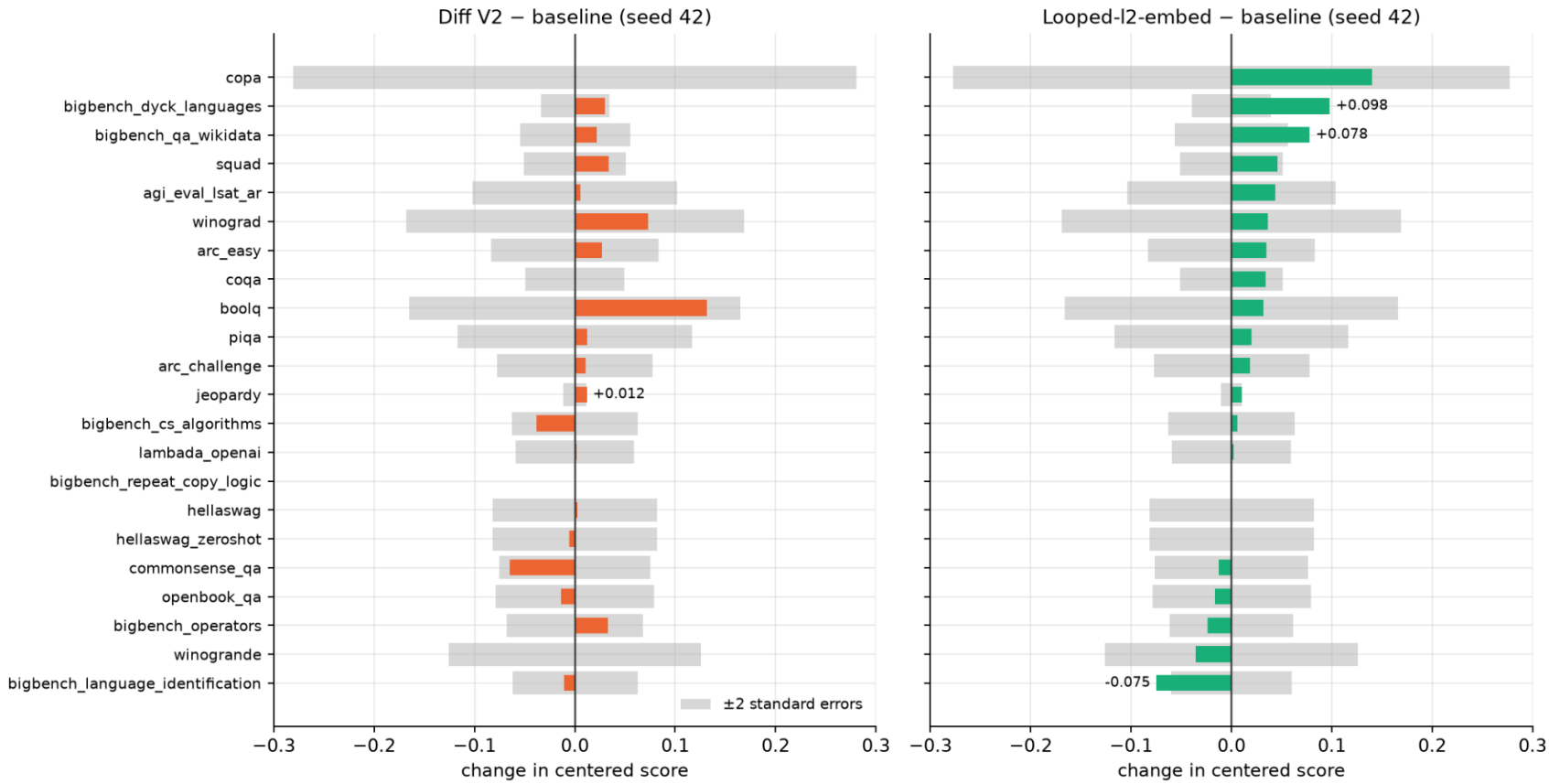


Figure 2.3. Change in each task's centered score against the baseline, at seed 42. The grey band is ±2 standard errors from sampling the task's examples (up to 500 per task); it does not include run-to-run noise. Labels mark the differences outside the band.

Analysis:

Most per-task differences are noise. Each task is scored on at most 500 examples, so a single run's centered score has a sampling error of a few hundredths, and much more on tasks with a high chance score such as BoolQ and COPA. Of 44 differences in Figure 2.3, only four fall outside the ±2 standard error band, and one of those (Diff V2 on Jeopardy, +0.012) is a move away from almost zero.

Looping's clearest gains are on BIG-bench Dyck Languages (+0.098) and QA Wikidata (+0.078), and its clearest loss is on Language Identification (−0.075). All three hold in direction on the 3-seed means (+0.050, +0.047 and −0.044).

The QA Wikidata gain comes from compute. Baseline-long gains almost as much on it (+0.069, mean of two runs), so this knowledge task rewards the extra training rather than the loops.

Dyck Languages is the one task where looping beats more training. It asks the model to close a sequence of nested brackets, a step-by-step task. Looped-l2-embed scores 0.162 against 0.088 for baseline-long at seed 42. This fits Part 1's expectation that repeated depth helps sequential reasoning, but it is one task and one comparison, so we treat it as a lead, not a result.

Diff V2 shows no consistent pattern. Its largest moves (BoolQ +0.132, Winograd +0.073, CommonsenseQA −0.065) are all inside the noise band, and it has only one seed.

Connection to the architectural research in Part 1: Looped-l2-embed shows a particularly relevant gain on BIG-bench Dyck Languages (0.162 versus 0.088 for Baseline-long). This task involves tracking nested bracket structures, potentially benefiting from the iterative representation refinement proposed in Part 1. In contrast, its Wikidata QA improvement largely disappears against Baseline-long, suggesting that additional training compute, rather than looping specifically, may explain that gain.

Diff V2 shows improvements on Winograd and BoolQ relative to the seed-42 baseline, potentially consistent with more selective contextual attention. However, these differences fall within the estimated evaluation uncertainty, so they do not establish a reliable task-specific advantage.

2.8 Why the looped model has a per-pass embedding

Our looped model adds one learned 768-dimensional vector to the hidden state at the start of each pass through the shared blocks (1,536 extra parameters, about 0.002% of the model). The vectors start at zero, so training begins from plain looping. Without them, the shared blocks can tell the two passes apart only through nanochat's per-layer scaling numbers, which rescale the hidden state but cannot push it in a new direction. We also trained looping without the vectors at the same three seeds:

Seed	Looped-l2 val bpb	Looped-l2-embed val bpb	Looped-l2 CORE	Looped-l2-embed CORE	CORE gap (embed − plain)
42	0.8380	0.8378	0.1752	0.1806	+0.0053
1	0.8375	0.8373	0.1740	0.1781	+0.0041
2	0.8375	0.8375	0.1576	0.1703	+0.0127
Mean	**0.8376**	**0.8375**	**0.1689**	**0.1763**	**+0.0074**

Table 2.7. Looping with and without the per-pass embedding (W&B project a3-archive, runs e4cyz84k, pzodxqp5 and qvumz6vt for plain looped-l2).

The two tie on val bpb (mean gap 0.0001). The embedding scores higher on CORE at every seed, by 0.0074 on average, but with three seeds and CORE's noise this is suggestive, not conclusive. Because it costs almost nothing, we kept it, and every “looped” result in this report uses it.

2.9 Cost

All runs ran on Modal, which bills H100 time per second at $3.95 per hour. Container time includes start-up, torch.compile, every validation pass and the final CORE evaluation.

Run	Training time	Container time	Tokens/s	MFU	Peak memory	Cost per run
Baseline (×3)	40.5 min	52.3 min	542K	41.6%	28.1 GiB	$3.45 ($3.43 and $3.53 for seeds 1 and 2)
Diff V2	51.3 min	64.1 min	427K	39.0%	32.8 GiB	$4.22
Looped-l2-embed (×3)	67.8 min	84.5 min	323K	41.6%	45.1 GiB	$5.57 ($5.54 and $5.65)†
Baseline-long (×2)	68.5 min	85.0 min	537K	41.3%	28.1 GiB	$5.59 ($5.58 for the repeat)†
Total (9 runs)	8.5 H100-hours	10.8 H100-hours				**≈$42.60**

Table 2.8. Time, throughput and cost. Times and rates are from the seed-42 runs; costs are container time × $3.95 per hour. †Three runs (looped seeds 1 and 2 and the baseline-long repeat) have no container time in the result files, so their cost uses the W&B run time plus the roughly 11 s of start-up seen in the other runs.

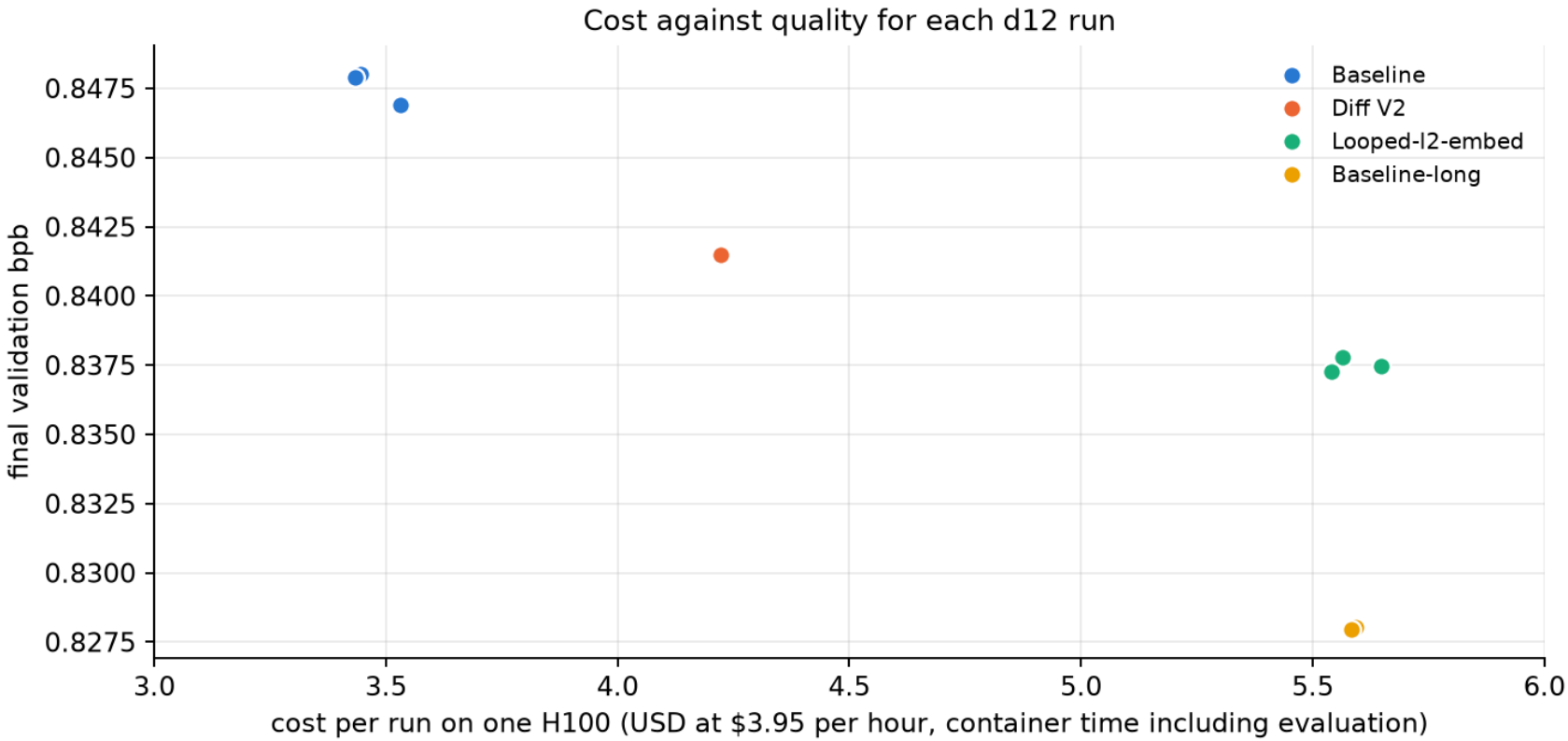


Figure 2.4. Final validation bpb against cost, one point per run.

Analysis:

- **Where the money went.** The training loop took 8.5 of the 10.8 H100-hours (79%); the rest is start-up, compilation, validation and CORE evaluation, about 12–17 minutes per run.
- **Cost of each change.** A Diff V2 run costs 22% more than a baseline run and a looped run about 60% more, because looping executes 22 layers. Looping also raises peak memory from 28.1 to 45.1 GiB (×1.6), since the activations of all 22 executed layers are kept for the backward pass.
- **The controls were worth it.** Baseline-long and the extra seeds were six of the nine runs, about $29 of the $42.60. Without baseline-long we could not have told that looping's gain comes from its extra compute.
- **Connecting these results to architectural research in part 1:** The computational costs reflect how the two changes interact with nanochat's implementation. Diff V2 preserves shared KV heads and Flash Attention 3 compatibility, but its

additional query projections and differential computations increase FLOPs per token. Its larger runtime increase and reduced MFU also suggest additional execution overhead. Looped-l2-embed demonstrates a different trade-off: it retains essentially the same Transformer weight count while increasing execution depth, resulting in more FLOPs per token and greater activation memory during backpropagation. Thus, looping provides parameter efficiency, but not necessarily training-compute or memory efficiency.

How we kept costs down:

- **Hardware.** An earlier team benchmark found the H100 fastest at a 2048-token context and about as cheap per run as the A100 and RTX PRO 6000, because Flash Attention 3 and higher throughput make up for its higher hourly price. Part 3 tests this choice in detail.
- **Smoke tests first.** Each configuration first ran 30 steps to catch configuration, memory and evaluation errors. The two smoke tests in the logs took 113–116 s of container time, about $0.13 each.
- **One GPU per run, run in parallel.** All runs in a batch trained at once on separate single GPUs, so a batch took about 1.5 hours of wall-clock time and no run waited for, or paid for, an idle multi-GPU machine.
- **Few repeats.** Three seeds for the baseline and looping, one for Diff V2. The accidental repeat of baseline-long cost $5.58 in duplicate training but gave us a direct measure of run-to-run noise; the launcher now re-attaches to runs it has already started.

2.10 Noise, tracking and limitations

- **Noise, measured.** The two identical baseline-long runs differ by 0.0001 bpb but 0.005 CORE. Across seeds, the baseline's range is 0.0011 bpb and 0.017 CORE (standard deviation 0.0006 and 0.0088), and Looped-l2-embed's is 0.0005 bpb and 0.010 CORE. bpb differences above about 0.002 are reliable here; CORE differences below about 0.02 are not.
- **Seeds.** Diff V2 and baseline-long ran at seed 42 only, so their CORE values are the least certain.
- **Data.** Baseline-long's 2.21B tokens repeat about 0.78B of the 1.43B in our 32 training shards (Section 2.2), while the other runs see each token once. Repeated data usually helps less than fresh data, so baseline-long's lead over looping is, if anything, understated.
- **One scale.** Every result is at d12 on 1.32B tokens. Section 2.11 discusses how they may change at d20.
- **Tracking.** Every run logged training loss, learning rate, throughput, MFU, validation bpb and per-task CORE to Weights & Biases, in project a3-final under group Part 2 with the tag part2. The plain looped-l2 runs and other exploratory ablations we dropped are in project a3-archive.

2.11 How these results may transfer to a larger run

- **Compare at equal compute, not equal tokens.** At d12, Diff V2's bpb gain disappears and looping's reverses once compute is matched. Any change considered for the d20 run should get the same check, so Part 5 trains a compute-matched d20 baseline alongside the final model.
- **Looping is unlikely to help a compute-limited run.** With plenty of data, unique weights beat repeated depth at equal compute. Looping suits settings where parameters or memory are the limit, not compute, and its one promising task (Dyck Languages) needs confirming at a larger scale.
- **Diff V2 deserves a compute-matched test at larger scale.** At d12 its gain is within noise once compute is accounted for, but Ye et al. (2025) report that differential attention's benefits grow with model size.
- **The gains should add.** The two changes act on different parts of the model, so at equal tokens we expect their gains to add, about 0.0167 bpb in total. Part 5 tests this at d20, where the combined model gains 0.0149 over its baseline, about 90% of that sum, but a baseline given the same compute still does better.
- **Overall conclusion:** Diff V2 remains promising at larger scales because its learned attention subtraction can be integrated into nanochat's existing Flash Attention implementation without expanding the KV cache. Whether this mechanism offers benefits beyond its additional compute requires further testing. Looped-l2-embed may be more appropriate when parameter storage is constrained, but additional computation is affordable. Its repeated execution increases effective depth without proportionally increasing weights, although the d12 results suggest this trade-off is less attractive when training compute is the primary constraint. Finally, the two modifications target complementary mechanisms—attention selection and iterative computation—so their combination could provide additional benefits. However, such gains are not necessarily additive, and larger-scale comparisons must again control for total training compute.

Part 3: Hardware Efficiency Curves

Different GPUs trade off compute, memory bandwidth, memory size, software support and price in different ways. In this part we re-ran both of our Part 2 ablations, **Diff V2** and **Looped-l2-embed**, on three GPUs and measured training time, cost, MFU and GPU telemetry for each run. The main findings:

- **The H100 is both the fastest and the cheapest per token.** It trains Diff V2 3.4× faster than the L40S and 1.5× faster than the B200, at $2.56 per billion tokens against $4.35 and $6.00.
- **Price per hour is a poor guide.** The L40S costs half as much per hour as the H100 but delivers only 29% of its throughput, so each token costs 1.7× more.
- **The newest GPU loses because of software, not silicon.** nanochat's Flash Attention 3 kernels do not run on Blackwell, so the B200 falls back to a much slower attention path. It runs at its power limit, yet its tensor cores are active only 23% of the time on Diff V2.
- **The hardware changes speed and cost, not what the model learns.** Final validation bpb agrees within 0.0007 across the three GPUs, about the size of the seed noise in Part 2.

3.1 Hardware choice

We chose three NVIDIA GPUs that differ in generation, product class and software support. Modal offers only NVIDIA GPUs, so the comparison is across generations and classes rather than vendors. Table 3.1 summarizes them.

GPU	Architecture (year)	Class	Rated peak bf16 (dense)	Measured peak bf16	Memory and bandwidth	Flash Attention 3	Modal price
H100 SXM 80GB	Hopper (2022)	Data-center training flagship	989 TFLOP/s	780 TFLOP/s (79%)	80 GB HBM3, 3.35 TB/s	Yes	$3.95 / GPU-h
L40S 48GB	Ada Lovelace (2023)	PCIe inference and graphics card	362 TFLOP/s	254 TFLOP/s (70%)	48 GB GDDR6, 0.86 TB/s	Yes	$1.95 / GPU-h
B200	Blackwell (2024)	Current data-center flagship	2,250 TFLOP/s	Not measured	About 180 GB HBM3e, about 8 TB/s	No (SDPA fallback)	$6.25 / GPU-h

Table 3.1. The three GPUs. Rated peaks are the values in nanochat's MFU table. Measured peaks are the best dense bf16 matrix multiply from our benchmark (`peak_bench.py`, square matrices up to 16,384), with the share of the rated peak in brackets. Memory figures are vendor specifications; prices are Modal's list prices.

Why these three:

- **H100 SXM** is the standard GPU for LLM training and our reference: all of Part 2 ran on it, and nanochat's kernels (Flash Attention 3) are written for it.
- **L40S** is the cheapest GPU per hour on Modal with enough memory for our model, and it is a different class of card: a PCIe card built for inference and graphics, with slower GDDR6 memory and about a third of the H100's compute. It tests whether a low hourly price turns into cheap training.
- **B200** is NVIDIA's current data-center flagship, with 2.3× the H100's rated bf16 peak and more than twice its memory. It tests whether a newer generation helps when the training code was not written for it.

Why one GPU of each: all of Part 2 ran on one GPU, so single-GPU runs are directly comparable with it, and they keep the comparison free of multi-GPU communication effects. Modal's Starter plan also allows only 10 GPUs at a time. Modal listed the B200 as unavailable on this plan, but our B200 jobs were scheduled after about 90 minutes in the queue. We also tried a B300: it crashed in `torch.compile`, because it needs CUDA 13.1 or newer and our torch 2.9.1 build uses CUDA 12.8, so it has no training run here. (With torch 2.15 and CUDA 13.2 our benchmark measured it at 1,831 TFLOP/s.)

3.2 Experimental setup

Every run is the d12 picochat from Part 2 trained for 2,520 steps of 524,288 tokens (1.32B tokens) on one GPU. The code (the team's nanochat branch), data and data order, seed (42), hyperparameters and total batch are the same on every GPU. Two things differ, and both are forced by the hardware:

- **Attention path.** On the H100 and L40S, attention runs with Flash Attention 3, which handles our sliding-window pattern (`SSSL`) efficiently. nanochat's Flash Attention 3 kernels have no Blackwell build, so on the B200 it falls back to PyTorch SDPA with an explicit sliding-window mask. The result is mathematically the same, but SDPA computes the full 2,048 × 2,048 attention and then masks it, so it does more work and uses far more memory.
- **Device batch.** Looped-l2-embed needs about 45 GB at a device batch of 32, more than the 44 GB the L40S can use, so it ran at a device batch of 16. The total batch stays at 524,288 tokens; the GPU just takes 16 smaller micro-steps per optimizer step instead of 8. Every other run used a device batch of 32.

What we measured:

- **Training time:** the training loop only. **Container time** adds start-up, `torch.compile`, 12 validation passes and the final CORE evaluation.
- **MFU (model FLOPs utilization):** the model's useful FLOPs per second, from nanochat's FLOP estimate, divided by the GPU's rated bf16 peak. This is the figure nanochat logs. **Achieved TFLOP/s** is the numerator alone, which can be compared across GPUs.
- **Cost:** Modal's list price × training-loop time (or × container time, including overhead), and cost per billion training tokens.

- **GPU telemetry:** power draw, memory and tensor-core activity, sampled by Weights & Biases during each run. Energy per billion tokens is mean GPU power during training × training time; it covers the GPU only, not the host.
- **Quality:** final validation bpb and CORE, to check that the hardware does not change what the model learns.

All runs are on Weights & Biases (a3-final workspace; Part 3 runs are tagged `part3`). The H100 rows are the seed-42 Diff V2 and Looped-l2-embed runs from Part 2 (Table 2.8), not new runs.

3.3 Results

Table 3.2 gives the main results for both models. Table 3.3 adds relative efficiency, energy and memory. Figures 4 to 8 show the training curves, throughput, cost breakdown and GPU telemetry.

Hardware	Training time	Container time	MFU (rated peak)	Achieved TFLOP/s	Tokens/s	Cost of training loop	Cost incl. overhead	Cost per 1B tokens	Final val bpb	CORE
Diff V2										
1× H100 SXM 80GB	**51.3 min**	**64.1 min**	39.0%	**385**	**427K**	**$3.38**	**$4.22**	**$2.56**	0.8415	0.173
1× L40S 48GB	177.0 min	204.3 min	30.8%	112	124K	$5.75	$6.64	$4.35	0.8408	0.153
1× B200 (SDPA)	76.0 min	93.4 min	11.6%	260	288K	$7.92	$9.73	$6.00	0.8409	0.180
Looped-l2-embed										
1× H100 SXM 80GB	**67.8 min**	**84.5 min**	41.6%	**412**	**323K**	**$4.46**	**$5.57**	**$3.38**	0.8378	0.181
1× L40S 48GB (device batch 16)	242.8 min	280.6 min	31.8%	115	90K	$7.89	$9.12	$5.97	0.8379	0.167
1× B200 (SDPA)	94.3 min	110.6 min	13.2%	296	233K	$9.82	$11.52	$7.43	0.8373	0.172

Table 3.2. Results on each GPU (one run each, seed 42). Bold marks the best value in each model's group for time, throughput and cost. Costs use Modal's list prices; the billed amounts are in Section 3.5. MFU on the B200 is against its rated peak and counts only the attention FLOPs the model needs, not the extra work SDPA does.

Model · GPU	*Speed vs. H100*	*Cost per token vs. H100*	*MFU vs. measured peak*	*Mean GPU power (limit)*	*GPU energy per 1B tokens*	*Peak memory (GiB)*	*Tensor-core pipe active*	*Device batch*

Diff V2								
H100	*1.00×*	*1.00×*	*49.4%*	*691 W (700 W)*	*0.45 kWh*	*32.8*	*42%*	*32*
L40S	*0.29×*	*1.70×*	*44.0%*	*330 W (350 W)*	*0.74 kWh*	*32.7*	*not reported*	*32*
B200	*0.67×*	*2.34×*	*–*	*969 W (1000 W)*	*0.93 kWh*	*95.3*	*23%*	*32*
Looped-l2-embed								
H100	*1.00×*	*1.00×*	*52.8%*	*692 W (700 W)*	*0.59 kWh*	*45.1*	*47%*	*32*
L40S	*0.28×*	*1.77×*	*45.3%*	*337 W (350 W)*	*1.03 kWh*	*24.0*	*not reported*	*16*
B200	*0.72×*	*2.20×*	*–*	*872 W (1000 W)*	*1.04 kWh*	*45.0*	*44%*	*32*

Table 3.3. Efficiency and telemetry. Speed is tokens per second relative to the same model on the H100; cost per token is relative to the H100's. Power, energy and tensor-core activity come from W&B system metrics while the GPU was training (utilization of at least 80%). Peak memory is PyTorch's peak allocation. The L40S does not report tensor-core activity, and the B200 has no measured peak.

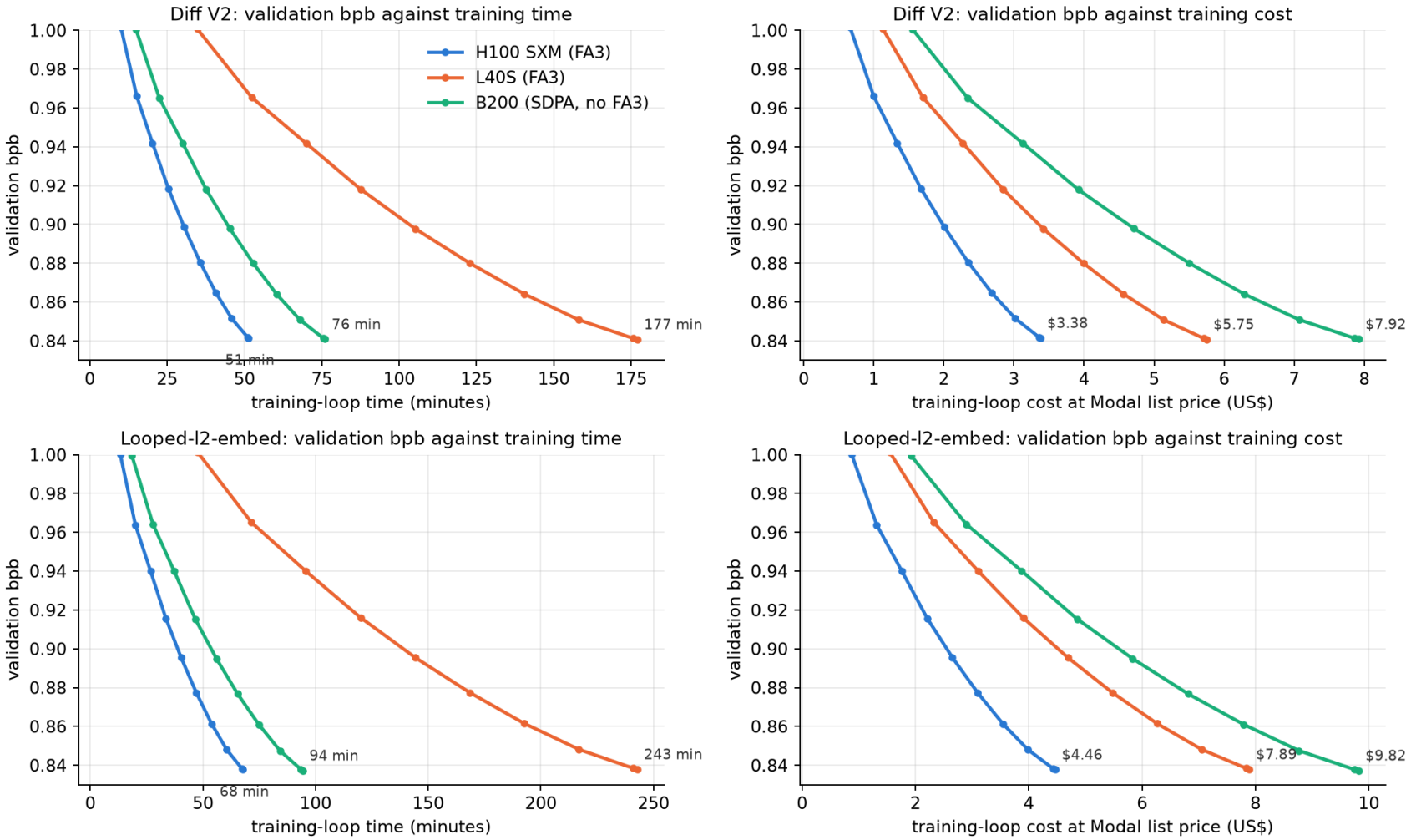


Figure 4. Validation bpb against training-loop time (left) and against training-loop cost at list price (right), for Diff V2 (top) and Looped-l2-embed (bottom). The H100 curve is furthest to the left in both: it reaches every bpb level soonest and most cheaply.

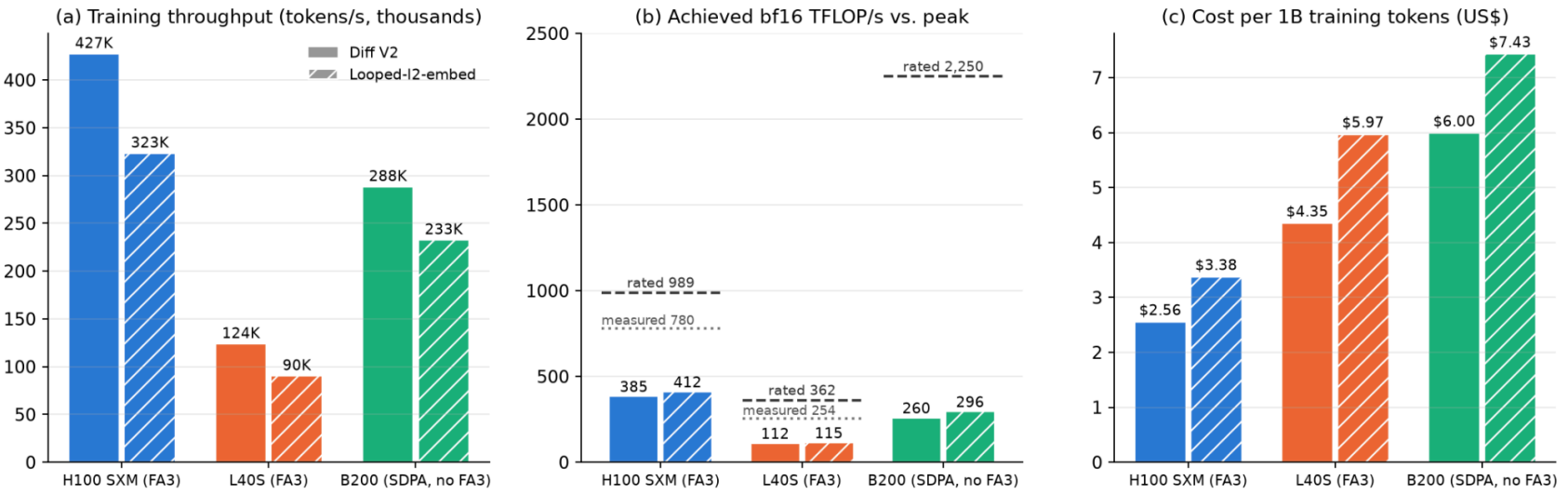


Figure 5. (a) Training throughput. (b) Achieved bf16 TFLOP/s, with each GPU's rated peak (dashed) and measured peak (dotted). (c) Cost per billion training tokens. Solid bars are Diff V2; hatched bars are Looped-l2-embed (device batch 16 on the L40S).

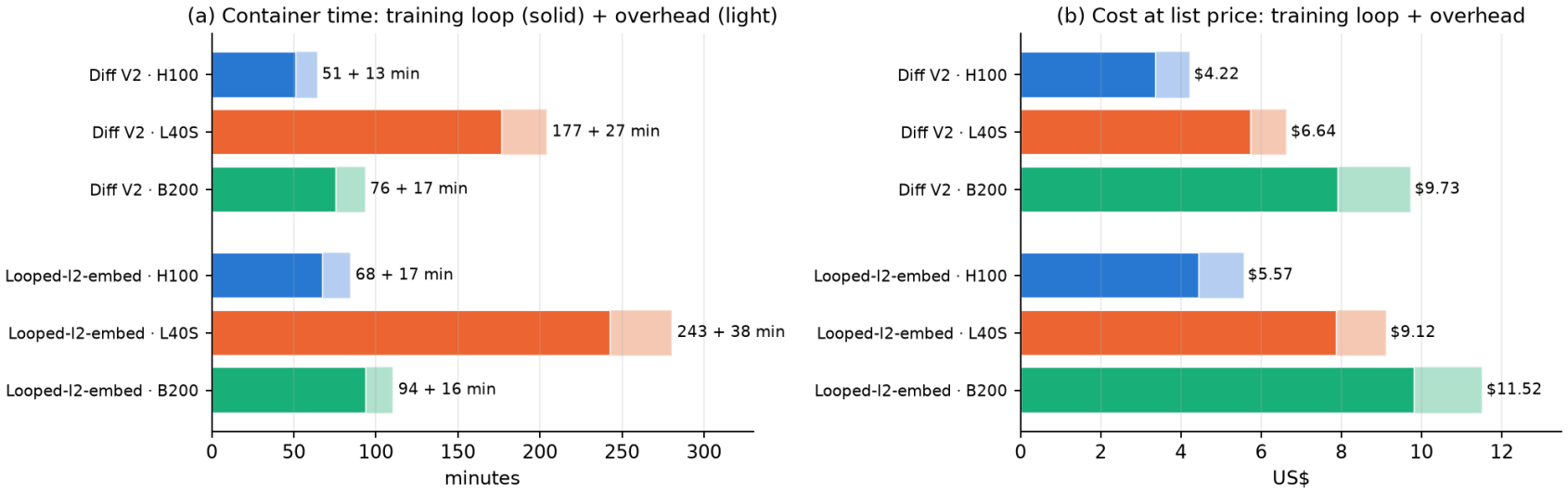


Figure 6. Where the time and money go: training loop (solid) plus overhead for start-up, compilation, validation and CORE (light).

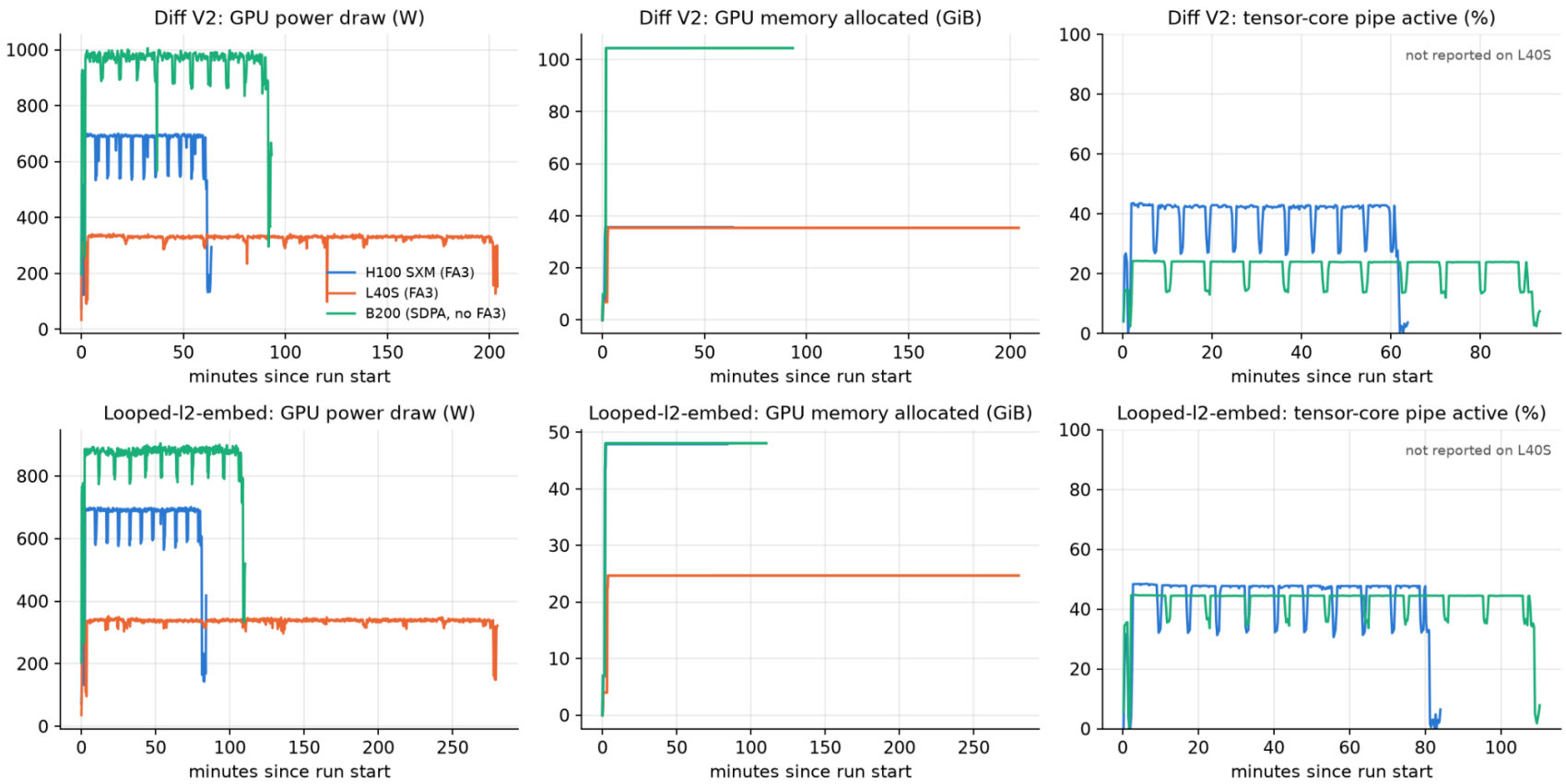


Figure 7. GPU telemetry from W&B over each run: power draw, memory allocated and tensor-core pipe activity. The regular dips are the validation passes every 250 steps.

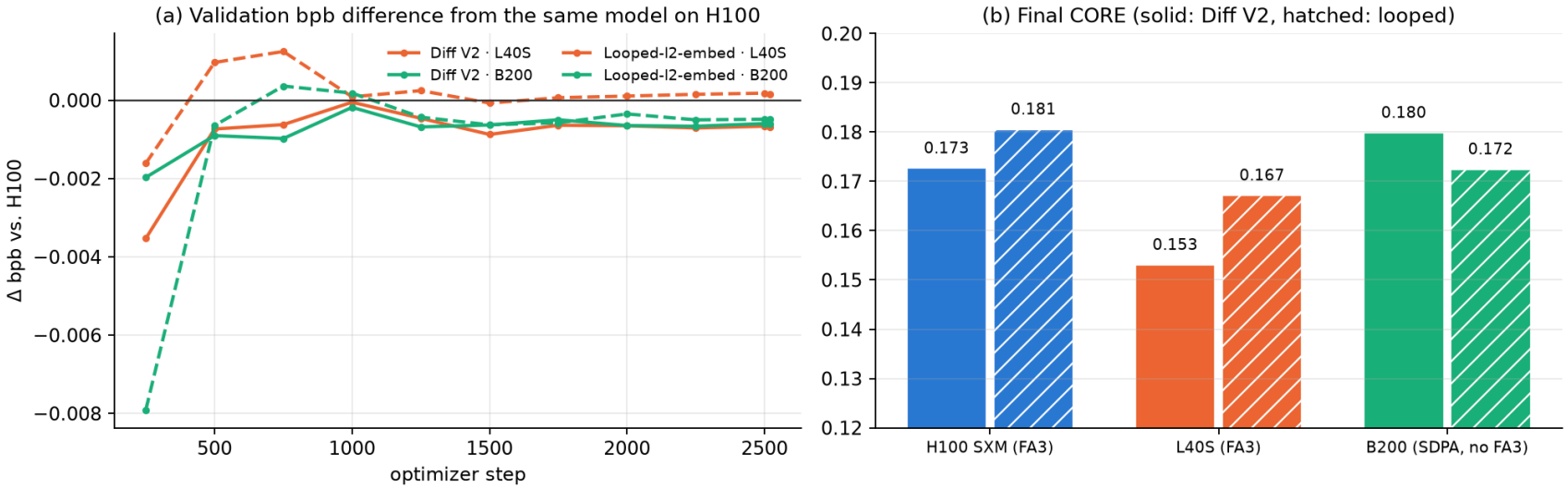


Figure 8. (a) Validation bpb on the L40S and B200 minus the same model's bpb on the H100, at each evaluation step. (b) Final CORE on each GPU.

3.4 Commentary

The H100 is both the fastest and the cheapest.

- On Diff V2 it trains 3.4× faster than the L40S (51 against 177 minutes) and 1.5× faster than the B200 (76 minutes). On Looped-l2-embed the gaps are 3.6× and 1.4×.
- It is also the cheapest per token: $2.56 per billion tokens on Diff V2, against $4.35 on the L40S (1.7×) and $6.00 on the B200 (2.3×). Figure 4 shows this holds at every point of training, not only at the end: for any target bpb, the H100 gets there first and for the least money.
- It also uses the least energy: 0.45 kWh of GPU energy per billion tokens on Diff V2, against 0.74 kWh on the L40S and 0.93 kWh on the B200.

A low hourly price does not mean cheap training.

- Cost per token is the hourly price divided by throughput. The L40S costs 49% of the H100's hourly price, so it would need at least 49% of the H100's throughput (about 210K tokens/s on Diff V2) to break even. It delivers 124K, or 29%.
- The shortfall matches the hardware. The L40S has 33% of the H100's measured compute (254 against 780 TFLOP/s) and 26% of its memory bandwidth (0.86 against 3.35 TB/s), and it achieves 29% of the H100's useful TFLOP/s (112 against 385). Our model is limited by both compute and memory traffic, so its speed falls between the two ratios.

- Lower power does not save energy either. The L40S draws half the H100's power (330 against 691 W) but runs 3.4× longer, so it uses 1.6× more energy per token.
- Memory size matters too. Looped-l2-embed did not fit on the L40S at a device batch of 32 and had to use 16. That still gave the same MFU as Diff V2 on the same card (31.8% against 30.8%), so the smaller micro-batch cost little speed here.

The newest GPU loses here because of software, not silicon.

- The B200 has 2.3× the H100's rated peak, but ran Diff V2 at 288K tokens/s against 427K, with 11.6% MFU against 39.0%, and cost 2.3× more per token.
- The telemetry shows why (Figure 7). The B200 ran at 97% of its 1,000 W power limit, so it was fully busy, yet its tensor cores (the units that do matrix multiplies) were active only 23% of the time on Diff V2, against 42% on the H100. Much of its time went into the non-matrix work of SDPA's masked full attention. Its peak memory for Diff V2 was 95 GiB, against 33 GiB on the H100.
- Diff V2 suffers more than Looped-l2-embed (23% against 44% tensor-core activity) because it doubles the number of query heads, so attention is a larger share of its work.
- A 30-step test on the H100 without Flash Attention 3 confirms that the attention path is the cause. Looped-l2-embed fell from 323K to 176K tokens/s and from 41.6% to 22.7% MFU, and Diff V2 ran out of memory on the H100's 80 GB.
- On the same attention path, the B200 is about 1.3× faster than the H100 (233K against 176K tokens/s on Looped-l2-embed). That is well below its 2.3× advantage in rated peak, so missing kernels are the main limit but not the only one. A short test is only a guide.

MFU compares a GPU with itself, not with other GPUs.

- MFU is relative to each chip's own peak. The L40S reaches 30.8% MFU on Diff V2 against the H100's 39.0%, which looks close, but in absolute terms the H100 does 3.4× more useful work per second (385 against 112 TFLOP/s).
- Rated peaks also flatter some cards. Our benchmark reached 79% of the H100's rated peak but only 70% of the L40S's. Against measured peaks, Diff V2's MFU is 49% on the H100 and 44% on the L40S, so both cards use their real capability similarly well; the L40S simply has less of it.
- On the B200, MFU understates the work actually done, because nanochat counts only the attention FLOPs inside each sliding window, while SDPA computes the full attention matrix.

The hardware changes speed and cost, not what the model learns.

- Final validation bpb agrees within 0.0007 across the three GPUs: 0.8408–0.8415 for Diff V2 and 0.8373–0.8379 for Looped-l2-embed. From step 1,000 on, every run stays within 0.001 of the same model on the H100 (Figure 8a).
- That spread is about the same as the spread between the baseline's three seeds in Part 2 (0.0011) and far smaller than the effect of the architecture changes (0.006–0.010). Different kernels and floating-point paths are not bit-identical, so small differences are expected.

- CORE varies much more: 0.153–0.180 for Diff V2 and 0.167–0.181 for Looped-l2-embed, with no consistent best GPU (Figure 8b). The same is true on one GPU type: Part 2's two identical baseline-long runs end 0.0001 bpb apart but 0.005 apart on CORE, and the baseline's three seeds span 0.017 on CORE. CORE differences of this size are noise, so CORE should be compared only across many runs.

Slower GPUs also pay more overhead. Validation and the final CORE evaluation run on the same GPU, so they take longer on slower cards. Overhead was 13–17 minutes per run on the H100, 16–17 on the B200 and 27–38 on the L40S (Figure 6).

3.5 Cost of this part

Item	Billed cost
B200 runs (Diff V2 and Looped-l2-embed)	$21.95
L40S Diff V2 run	$7.08
L40S Looped-l2-embed run	$9.71
Total spent on Part 3	**$38.74**

Table 3.4. Modal's billed costs for Part 3. The two H100 reference runs were paid for in Part 2 and are not counted here.

- **Billed against list price:** the B200 runs were billed $21.95, against $21.25 from list price × container time (+3%). The L40S runs were billed $16.79, against $15.76 (+7%). The difference is CPU and memory charges, which make up a larger share of the bill on a cheap GPU.
- **Wall-clock time:** the runs overlapped, so Part 3 took about 6 hours from launch to the last result. That includes about 90 minutes of queueing for the B200s.
- **Spread of costs:** the most expensive run (Looped-l2-embed on the B200, $11.52) costs 2.7× the cheapest (Diff V2 on the H100, $4.22), for models of about the same quality.

3.6 Recommendation and how this transfers

- **For the final model (Part 5), use H100s, with 8 GPUs in one container.** They give the shortest time, the lowest cost per token and the lowest energy per token, because nanochat's kernels are built for them.
- **The B200 becomes worth it only with Blackwell kernels.** At $6.25 against $3.95 per hour, it must be at least 1.58× faster than the H100 to be cheaper per token. On a matched attention path it was only about 1.3× faster, and without Flash Attention

3 Diff V2's memory use nearly triples. A Blackwell attention kernel that supports sliding windows (or switching to full-context attention) would be needed first, and the result should be re-measured.

- **The L40S is not a good training card for this model,** but its low price and 48 GB of memory could suit work that does not need high throughput, such as evaluation or inference.
- **At larger scale the H100's lead should hold.** A larger model spends a larger share of its time in matrix multiplies, which favours GPUs with high compute and fast memory. Multi-GPU runs add communication over NVLink, which the H100 SXM and B200 have and the PCIe L40S lacks. We measured only single-GPU runs, so the multi-GPU numbers should be checked on the actual Part 5 run.

Part 4: Extending the Context Window

We extended Picochat's training context from 512 to 2048 tokens and compared the checkpoints before and after the extension on a new retrieval task. We also evaluated the Part 2 baseline, trained at 2048 tokens throughout, to compare the two sequence-length schedules at the same total training-token budget.

4.1: Training Setup

Saved checkpoint	Sequence length	Steps in stage	Tokens in stage
A | step 1260	512	1260	660,602,880
B | step 2520	2048	1260	660,602,880

Table 4.1. The two stages each use half of the 1,321,205,760-token training budget.

We used the depth-12 Picochat baseline and the same tokenizer and total token budget as in Part 2. We trained on ClimbMix shards 0 to 31, with a separate shard for validation. Both stages used seed 42 and 524,288 tokens per optimizer step. Table 4.1 shows how the budget was split between the two checkpoints.

Starting at 128 or 256 tokens would give Picochat less context in the first stage and require a sixteenfold or eightfold jump to reach 2048. In GrowLength's comparison, training loss rose sharply after the largest jump and changed more smoothly in schedules that passed through intermediate lengths (Jin et al., 2023, Section 4.5). We chose 512 as a compromise. It reduces the first-stage window to 1/4 of the target while limiting the size of our single transition.

In Shortformer's WikiText-103 experiment, using 512-token sequences for 100 of 205 epochs and then increasing to 3072 yielded a final validation perplexity of 18.10, compared with 18.65 for the full-length baseline (Press et al., 2021, Table 2). We used this near-half split as a precedent and set the switch at 50% of our token budget before training. Each stage processed 660.6 million tokens.

At step 1260, checkpoint A's data loader was still on the first pass through ClimbMix, with its saved position in shard 20. We restored A's weights and optimizer state, continued the same 2520-step learning-rate schedule, and resumed reading from the saved position, packing the text into 2048-token sequences. After 1260 additional steps at the sequence length of 2048, we saved checkpoint B at step 2520.

For an additional comparison, we reused the trained seed-42 d12-baseline checkpoint from Part 2. It used the same model architecture, tokenizer, ClimbMix training shards, optimizer settings, and learning-rate schedule as our run. Its training sequence

length was 2048 throughout all 2520 steps. Both this baseline and checkpoint B therefore processed 1,321,205,760 tokens, while A was saved halfway through that budget. The baseline was evaluated without further training.

Figure 4.1 shows the logged training loss and validation bits per byte (BPB).

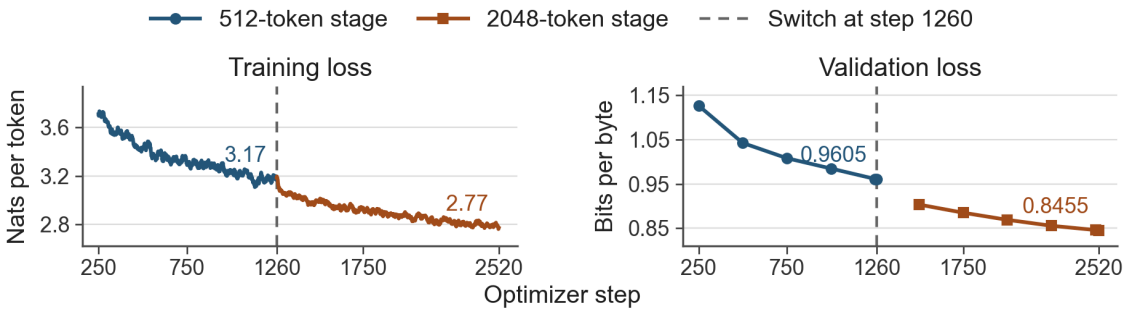


Figure 4.1. Loss curves from step 250. At step 0, validation BPB was 3.1727.

The logged training loss is a moving average of next-token cross-entropy. It fell from 3.43 at step 500 to 3.17 by checkpoint A, and from 2.99 at step 1500 to 2.77 by checkpoint B. Validation BPB also fell within each stage, from 1.0426 to 0.9605 at 512 tokens and from 0.9034 to 0.8455 at 2048 tokens. Lower BPB indicates better prediction on held-out text. The left panel reports training loss in nats per token, and the right panel reports validation loss in bits per byte. To calculate validation BPB, we packed text from the same held-out ClimbMix shard into 512-token sequences in the first stage and 2048-token sequences after the switch at step 1260.

4.2: Evaluation Design and Checks

We designed a task in which the model retrieves a locker's access code from its input. In the single-record version, each prompt contains one sentence assigning a code word to a locker. The final sentence asks the model to repeat that locker's code. We place unrelated text before and after the locker sentence to control the prompt's length and the code's position. The prompts are generated independently of ClimbMix from handwritten text and randomized code assignments.

> For locker 42, the access code is **amber**.
> *[Unrelated filler text]*
> As written above, the access code for locker 42 is ___

Single-record example with most filler text omitted. The model predicts the token at the blank.

For each prompt, we select four different candidate code words, each represented by one token under our tokenizer. We randomly choose one to place in the single-record prompt, which contains exactly one of these four words. The model assigns a prediction score to each token in its vocabulary, indicating how likely that token is to come next. The evaluation code compares the prediction scores of the four candidate words and chooses the word with the highest score. We mark the prompt correct if the chosen word matches the code given for the requested locker.

We measure accuracy as the proportion of prompts answered correctly. Choosing uniformly at random among the four candidates would give 25% expected accuracy.

The four-record version tests retrieval when several plausible code words appear in the input, drawing on RULER's use of multiple pieces of information in retrieval tasks (Hsieh et al., 2024). Each prompt contains four sentences assigning different code words to different lockers, so all four candidate codes appear once. The model must select the code associated with the locker asked about in the final sentence. The requested locker and the order of records vary across prompts.

> For locker 17, the access code is violet.
> For locker 93, the access code is maple.
> For locker 42, the access code is silver.
> For locker 58, the access code is coral.
> As written above, the access code for locker 42 is ___

Four-record example with filler text omitted. Locker 42 is requested, so the correct code is silver.

We varied the total input length and the distance from the requested code to the end of the prompt to test whether the model can use information at different positions (Liu et al., 2024). Distance counts the tokens after that code up to the final input token. For 512-token prompts, the tested distances were 128 and 384 tokens. For 1024-token prompts, they were 128, 384, and 768. For 2048-token prompts, they were 128, 384, 768, and 1536.

Each of these nine combinations has 128 prompts for each task version. A, B, and the Part 2 baseline receive identical prompts and candidate codes within each task version. At evaluation, we set the maximum input length to 2048 for all three, allowing us to test A beyond its 512-token training length.

We used two edits to the single-record prompts to examine whether this evaluation depended on the code supplied in the input. We ran both checks using checkpoint B.

In the first check, we replaced the stated code with "unknown". For example, "the access code is amber" became "the access code is unknown". The scoring program continued to compare the four original code words, including amber and excluding unknown. We

measured how often it still selected amber after that information had been removed from the prompt. The 25% guessing reference therefore applies to selecting the removed original code.

In the second check, we replaced the stated code with a different one of the four candidate words and measured how often the model selected that replacement. For example, replacing amber with silver made silver the expected answer. The final question and the four candidate words stayed the same in both checks.

After we replaced the code with "unknown", B selected the removed original code in 28 to 35 of the 128 prompts at each tested combination of input length and code distance. This was 21.9% to 27.3%, close to the 25% reference for guessing the original code. When we replaced it with a different candidate word, B selected the replacement in all 128 prompts at every tested combination. The results matched our expectations in both checks, supporting the validity of our retrieval evaluation.

For the four-record task, we estimated uncertainty for two accuracy differences: B minus A, and B minus the Part 2 baseline. For each identical prompt, we recorded 1 for a correct answer and 0 for an incorrect answer. We subtracted the reference model's correctness value from B's value, then averaged the 128 differences. We used their variation to calculate an approximate 95% paired confidence interval (CI), following Miller (2024, Section 4.2). Positive differences favor B, and negative differences favor the reference. Each interval is calculated separately for one condition and describes variation across the generated prompts for that fixed checkpoint pair.

The single-record and four-record prompt generators and their shared scoring function are in a3/nanochat/scripts/part4_eval.py. Commands for repeating the evaluations are in a3/part4/README.md, and the saved four-record predictions and paired summaries are in a3/part4/results/.

4.3: Checkpoint A and B Results

Input tokens	Code distance	Single record A	Single record B	Four records A	Four records B
512	128	128 (100.0%)	128 (100.0%)	46 (35.9%)	44 (34.4%)
512	384	128 (100.0%)	128 (100.0%)	11 (8.6%)	28 (21.9%)
1024*	128	128 (100.0%)	128 (100.0%)	44 (34.4%)	63 (49.2%)
1024*	384	128 (100.0%)	128 (100.0%)	29 (22.7%)	30 (23.4%)
1024*	768	100 (78.1%)	128 (100.0%)	1 (0.8%)	15 (11.7%)

Input tokens	Code distance	Single record A	Single record B	Four records A	Four records B
2048*	128	128 (100.0%)	128 (100.0%)	45 (35.2%)	85 (66.4%)
2048*	384	104 (81.3%)	128 (100.0%)	18 (14.1%)	38 (29.7%)
2048*	768	45 (35.2%)	128 (100.0%)	8 (6.3%)	20 (15.6%)
2048*	1536	68 (53.1%)	128 (100.0%)	22 (17.2%)	31 (24.2%)

Table 4.2. Correct answers out of 128 prompts, with percentage accuracy in parentheses. Input length and code distance are measured in tokens. The asterisk marks inputs longer than A's 512-token training sequences.

Table 4.2 shows that B answered all 128 single-record prompts correctly at each of the nine tested combinations of input length and code distance. Both checkpoints reached 100% on the 512-token prompts. At an input length of 2048 and a code distance of 768, A scored 35.2% and B scored 100%. B's 100% accuracy motivated the four-record version and the two checks using edited prompts described in Section 4.2. A single-record prompt can be solved by finding the only candidate code in the input. With four records, success also requires matching a code to the requested locker. On that task at the same length of 2048 and distance of 768, A scored 6.3% and B scored 15.6%.

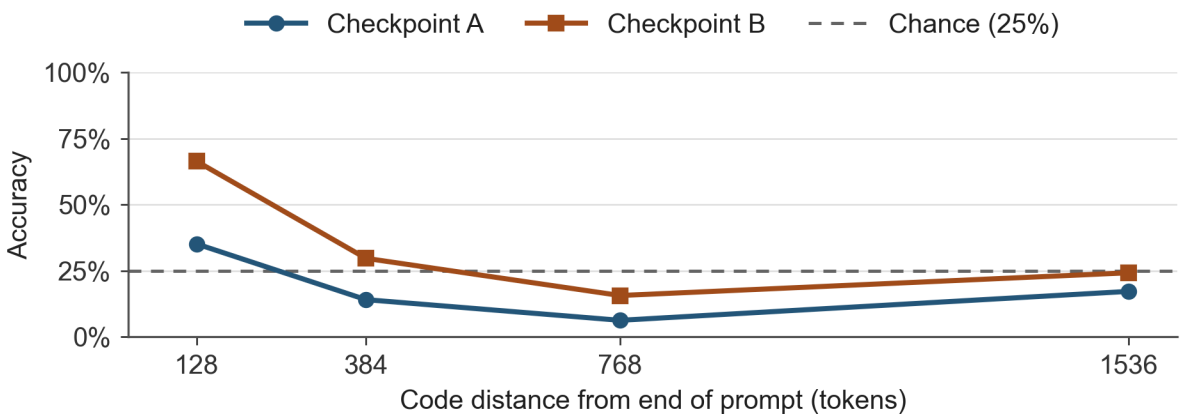


Figure 4.2. Four-record accuracy at an input length of 2048 tokens. At each code distance, A and B receive the same 128 prompts. The dashed line marks 25% accuracy from uniform guessing.

Figure 4.2 shows how the four-record results change with code distance in a 2048-token prompt. At a distance of 128, B scored 66.4% compared with A's 35.2%. B's accuracy fell to 15.6% at a distance of 768 and was 24.2% at 1536. Both of these longer-distance scores were at or below the 25% guessing reference, despite B's perfect single-record performance.

At an input length of 2048 and a code distance of 128, B answered 40 more prompts correctly, a gain of 31.3 percentage points (95% CI 21.6 to 40.9). At an input length of 1024 and a distance of 384, the gain was 0.8 percentage points (95% CI -10.2 to 11.8). In this second comparison, B answered only one more prompt correctly. The interval includes zero, corresponding to equal accuracy, so the result provides little evidence of an advantage for B on that type of prompt.

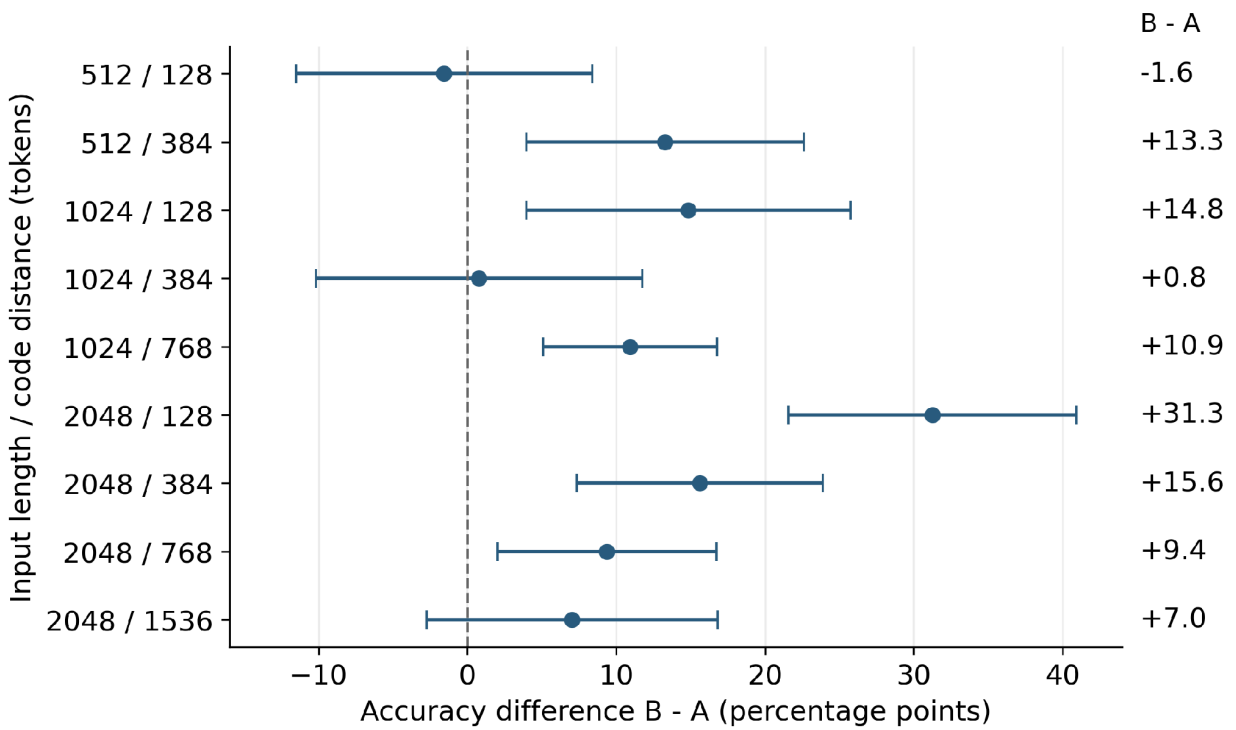


Figure 4.3. Four-record accuracy differences across all nine conditions. Points show B minus A; horizontal bars show approximate 95% paired confidence intervals. The dashed vertical line marks equal accuracy. Row labels give input length / code distance in tokens.

Figure 4.3 shows B's four-record accuracy minus A's, with approximate 95% confidence intervals calculated from the 128 test prompts in each condition. Six intervals lie entirely above zero, providing evidence of higher accuracy for B in those conditions. The remaining three include zero and do not provide clear evidence of a difference between A and B.

Overall, checkpoint B performed better than A on our retrieval task: it answered all single-record prompts correctly and achieved higher four-record accuracy in eight of the nine tested conditions. However, with 2048-token inputs, B's four-record accuracy was

only 15.6% at a code distance of 768 and 24.2% at 1536, both below the 25% guessing reference. B's perfect single-record accuracy did not carry over to the four-record task.

B received 1260 additional optimizer steps using 2048-token sequences, following A's training with 512-token sequences. The A-to-B comparison therefore measures the combined effect of additional training and longer training sequences.

Section 4.4 compares B with the Part 2 baseline at the same total number of optimizer steps and training tokens to examine how the two training approaches performed at an equal token budget.

4.4: Comparison with the Part 2 Baseline

The Part 2 baseline answered all 128 single-record prompts correctly in every condition, matching B. Table 4.3 compares their four-record results on the same prompts, candidate words, and expected answers.

Input tokens	Code distance	Checkpoint B	Part 2 baseline	B - baseline (pp)
512	128	44 (34.4%)	66 (51.6%)	-17.2
512	384	28 (21.9%)	30 (23.4%)	-1.6
1024	128	63 (49.2%)	75 (58.6%)	-9.4
1024	384	30 (23.4%)	48 (37.5%)	-14.1
1024	768	15 (11.7%)	26 (20.3%)	-8.6
2048	128	85 (66.4%)	95 (74.2%)	-7.8
2048	384	38 (29.7%)	74 (57.8%)	-28.1
2048	768	20 (15.6%)	33 (25.8%)	-10.2
2048	1536	31 (24.2%)	42 (32.8%)	-8.6

Table 4.3. Four-record correct answers out of 128 prompts, with percentage accuracy in parentheses. The last column gives B minus the Part 2 baseline in percentage points (pp). Negative values favor the baseline. Input length and code distance are measured in tokens.

The Part 2 baseline had higher four-record accuracy in all nine conditions. At an input length of 2048 and a code distance of 384, it answered 74 prompts correctly (57.8%), compared with B's 38 (29.7%). The B-minus-baseline difference was -28.1 percentage points (95% CI -39.6 to -16.6). At the same input length and a distance of 128, the difference was -7.8 percentage points (95% CI -19.0 to 3.4). That interval includes zero, so this condition provides less clear evidence of a difference.

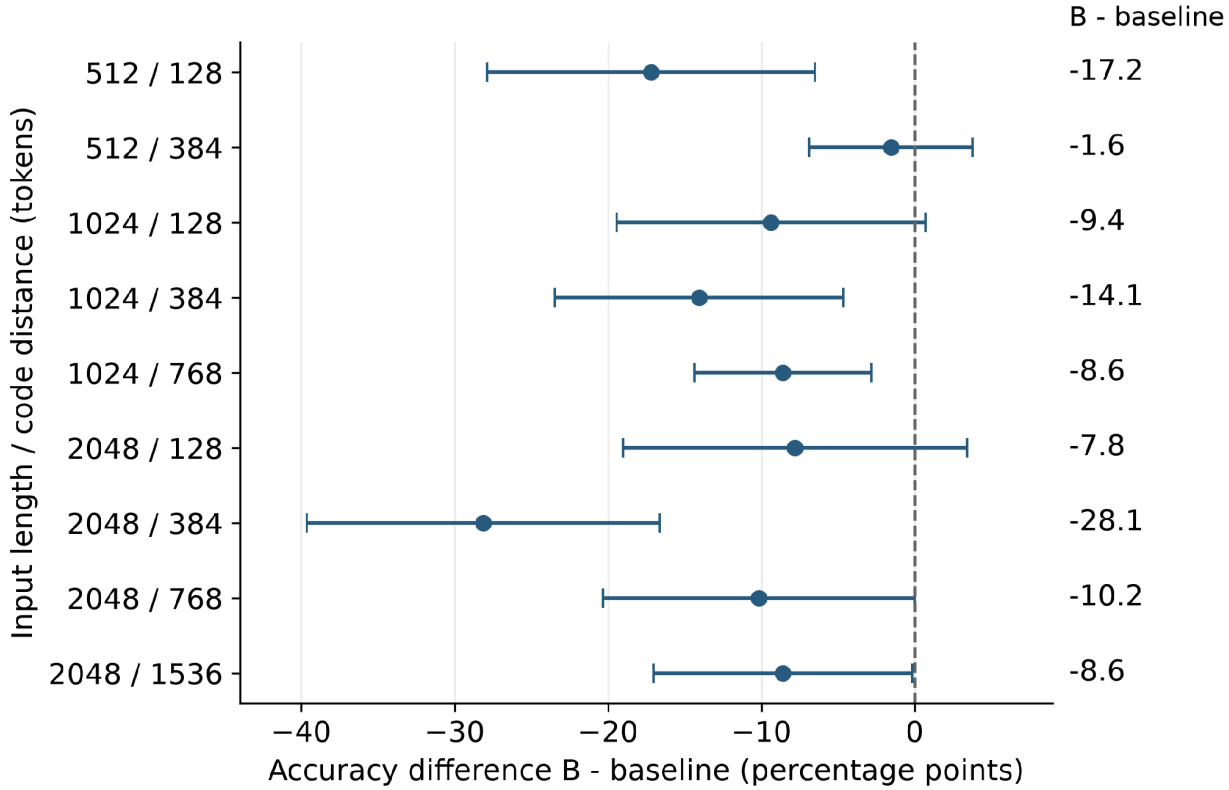


Figure 4.4. Four-record accuracy differences between B and the Part 2 baseline across all nine conditions. Points show B minus baseline; horizontal bars show approximate 95% paired confidence intervals. Negative differences favor the baseline, and the dashed line marks equal accuracy. Row labels give input length / code distance in tokens.

Figure 4.4 shows five intervals entirely below zero and four that include zero. The baseline's numerical advantage is therefore more clearly supported in some conditions than in others.

Both the baseline and checkpoint B were trained for 2520 optimizer steps and processed 1,321,205,760 tokens. The baseline used 2048-token sequences throughout, while checkpoint B used 512-token sequences for 1260 steps and 2048-token sequences for another 1260 steps. The baseline achieved higher four-record accuracy than checkpoint B in all nine tested conditions. At this training-token budget, training at 2048 throughout therefore gave better four-record retrieval accuracy than B's 512-to-2048 training.

4.5: Conclusion

Continuing from A to B improved four-record retrieval accuracy in eight of the nine tested conditions. Six of the nine paired 95% confidence intervals for B minus A were entirely above zero, providing evidence of an advantage for B in those conditions, and the remaining three included zero. This comparison measures the combined effect of 1260 additional optimizer steps and increasing the training sequence length from 512 to 2048.

The single-record results showed that B could recover an isolated code at every tested input length and distance, including conditions where A's accuracy was lower. With four competing records, B's accuracy varied with code distance and remained below the 25% guessing reference at the two longest tested distances in 2048-token inputs. Together, the two task formats show that the continuation improved retrieval, while selecting the code associated with the requested locker remained more difficult than recovering the only candidate code present in the input. The single-record evaluation alone would have hidden this difference.

The Part 2 baseline provides an additional comparison of the training schedules. At the same total training-token budget, training at 2048 throughout produced higher four-record accuracy than B in all nine conditions. Under the tested budget, this approach therefore performed better on the four-record task than our 512-to-2048 schedule.

Part 5: Final Nanochat

For the final model we trained a d20 nanochat with both of our architecture changes, Diff V2 and Looped-l2-embed, on 5.22B tokens using 8 H100 GPUs. To judge what the changes were worth, we also trained a plain d20 baseline on the same tokens and a second baseline given the same total compute as the final model. The main findings:

- **The final model is our best at a fixed token budget.** It reaches 0.7210 validation bpb and a CORE of 0.266, against 0.7359 and 0.258 for the d20 baseline. All the d20 models beat the GPT-2 checkpoint's CORE of 0.2565.
- **Scale did most of the work.** Going from d12 to d20 improved bpb by 0.1117; adding both changes at d20 improved it by a further 0.0149. The first cost 15× the compute, the second 2.14×.
- **The changes are not the best use of compute.** A d20 baseline trained for the same total FLOPs reached 0.7166 bpb, better than the final model's 0.7210, with CORE level within noise. Part 2 found the same at d12.
- **Our scaling prediction was too pessimistic.** Kaplan et al.'s size-only exponent predicted 0.764 bpb for the d20 baseline; it reached 0.7359, because tokens grew along with parameters. Kaplan's compute law, applied after the fact, brackets the result.
- **The bigger model knows far more facts.** The d20 model answered 43 of 50 factual questions, against 23 and 21 for two d12 models, and it was never wrong where a d12 model was right.

5.1 Final configuration

The handout asks for a final model that includes our architecture changes. We kept the two changes from Part 2 that lowered validation bpb the most at a fixed token budget and combined them. We set this rule after seeing Part 2's results, not in advance. Table 5.1 summarizes the configuration.

Choice	Decision	Why
Architecture	Diff V2 + Looped-l2-embed: 1 prelude block, 18 core blocks run twice with a learned per-pass embedding, and 1 coda block, for 38 executed layers	At d12 these were the two changes that lowered bpb most: Looped-l2-embed by 0.0101 (3 seeds) and Diff V2 by 0.0066 (1 seed). They change different parts of the model, so we expected their gains to add. Part 2 did not train the two together at d12.
Size	d20: width 1,280, 10 attention heads, 435M scaling parameters	d20 is nanochat's default depth and fit our budget. A 4×H100 test of this model ran at 265K tokens/s and 41% MFU, which projected about 3 hours and $110 on 8 GPUs. A d24 model would need about 2.8× the compute (about $235 for the training loop at list price, against $84 for d20), which we

		chose not to spend.
Data	5.22B tokens of ClimbMix: 9,960 steps × 524,288 tokens, reaching shard 109 of the 125 downloaded	nanochat's rule of 12 tokens per scaling parameter, the same ratio as our d12 runs, so both sizes are trained to the same point relative to their size.
Tokenizer	The team tokenizer from Part 2 (32,768 tokens, trained on 1B characters); seed 42	Keeps the d12 and d20 results directly comparable.
Hardware	8×H100 in one container, Flash Attention 3, device batch 8	Part 3: the H100 gave the lowest cost per token and the shortest training time. Blackwell GPUs have no Flash Attention 3 build, and without it Diff V2 nearly triples its memory use.

Table 5.1. Final configuration (run tag d20-combined). In this part, "looped" always means the version with per-pass embeddings (Looped-l2-embed).

A note on the full dataset. The handout asks for training on the full pre-training dataset, which we did not do. We trained on 110 of ClimbMix's 6,542 shards, about 1.7% of the data. All of it would be about 59 times as many tokens and roughly $5,000 at this run's rate. We followed nanochat's 12 tokens per scaling parameter instead, which is the same order as the roughly 20 tokens per parameter that Hoffmann et al. (2022) found compute-optimal. More data would probably have lowered bpb further, but we did not measure by how much.

5.2 What we trained

We trained three d20 models, all on 8×H100 with the same code, data order and hyperparameters, so the number of GPUs does not affect the comparison between them:

- **d20 baseline:** the unchanged model on the same 5.22B tokens as the final model.
- **d20 combined:** the final model.
- **d20 baseline at equal compute:** the baseline trained for the final model's total FLOPs (21,328 steps, 11.18B tokens, 3.23e19 FLOPs). A data cap kept it to the same first 110 shards, so it read them about 2.1 times and saw no more unique data. It tests whether the final model's extra compute would have been better spent on more training steps.

For comparison we use the d12 picochats from Part 2. The baseline and Looped-l2-embed have three seeds each; Diff V2 has one (seed 42), so its gains are measured against the seed-42 baseline. Part 2 also trained baseline-long, the d12 baseline given Looped-l2-embed's compute, which is the d12 counterpart of our equal-compute d20 run.

5.3 Results

Model	Runs	Training tokens	FLOPs per token	Total compute vs. same-depth baseline	Val bpb	Δ bpb	CORE	Δ CORE
d12 picochat (Part 2)								
Baseline	3 seeds	1.32B	7.60e8	×1.00	0.8476	–	0.154	–
Diff V2	1 seed	1.32B	9.02e8	×1.19	0.8415	−0.0066*	0.173	+0.012*
Looped-l2-embed	3 seeds	1.32B	12.74e8	×1.68	0.8375	−0.0101	0.176	+0.023
Baseline-long (equal compute to Looped)	2 identical runs	2.21B	7.60e8	×1.68	0.8280	−0.0196	0.167	+0.013
d20 nanochat (this part)								
Baseline	1	5.22B	28.86e8	×1.00	0.7359	–	0.2575	–
Combined (final model)	1	5.22B	61.80e8	×2.14	**0.7210**	−0.0149	**0.2662**	+0.0087
Baseline, equal compute	1	11.18B (110 shards, read about 2.1×)	28.86e8	×2.14	0.7166	−0.0193	0.2631	+0.0056

*Table 5.2. Results. Δ is against the baseline at the same depth (at d12, the 3-seed mean); lower bpb and higher CORE are better. *Against the seed-42 baseline (0.8480 bpb, 0.161 CORE), since Diff V2 has only that seed. Baseline-long repeated about 0.78B of its tokens, because Part 2's runs used 32 data shards.*

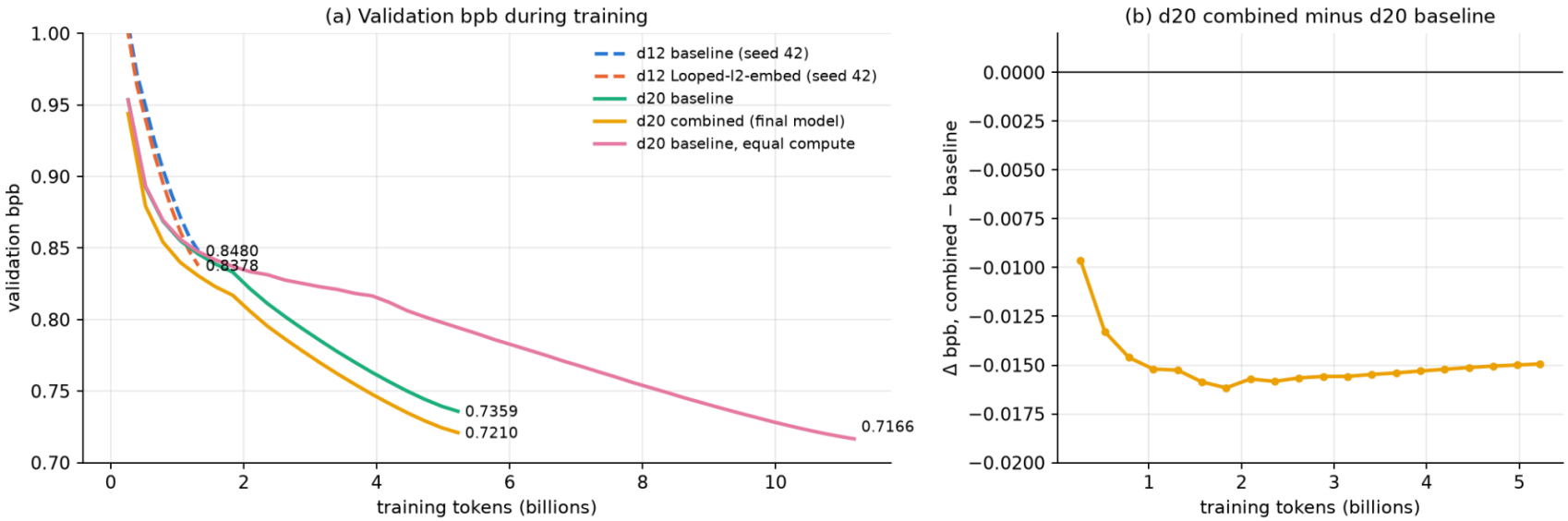


Figure 5.1. (a) Validation bpb against training tokens. The equal-compute baseline uses its own, longer learning-rate schedule, so its curve only becomes comparable at the end. (b) The final model's lead over the d20 baseline at each evaluation.

Table 5.3 puts each effect side by side. CORE differences below about 0.009, the standard deviation across the three d12 baseline seeds, are within noise; Part 2's two identical baseline-long runs differed by 0.005.

Change	Evidence	Δ bpb	Δ CORE	Compute	What it shows
Size and data	d12 baseline → d20 baseline	−0.1117	+0.104	×15.0	By far the largest effect on both metrics.
Diff V2	d12, 1 seed	−0.0066	+0.012	×1.19	A clear bpb gain (identical runs differ by under 0.0002 bpb); the CORE gain is at the edge of noise.
Looped-l2-embed	d12, 3 seeds	−0.0101	+0.023	×1.68	The larger gain on both metrics, at the larger cost.
Both, expected	Sum of the two d12 gains	−0.0167	–	–	What d12 would show if the gains simply added. Part 2 has no joint d12 run.
Both, measured	d20, 1 run each	−0.0149	+0.0087	×2.14	About 90% of the expected gain carried over to d20. The CORE gain is within noise.

More steps instead of looping	d12 baseline-long vs. Looped-l2-embed	−0.0196 vs. −0.0101	+0.013 vs. +0.023	×1.68 each	The same compute spent on more tokens beat looping on bpb by 0.0095. Looping's 0.010 CORE lead is about one standard error, so not conclusive.
More steps instead of both changes	d20 equal-compute baseline vs. combined	−0.0193 vs. −0.0149	+0.0056 vs. +0.0087	×2.14 each	More steps won on bpb by 0.0043; the CORE difference is within noise.

Table 5.3. The impact of each change. Δ values are against the baseline at the same depth.

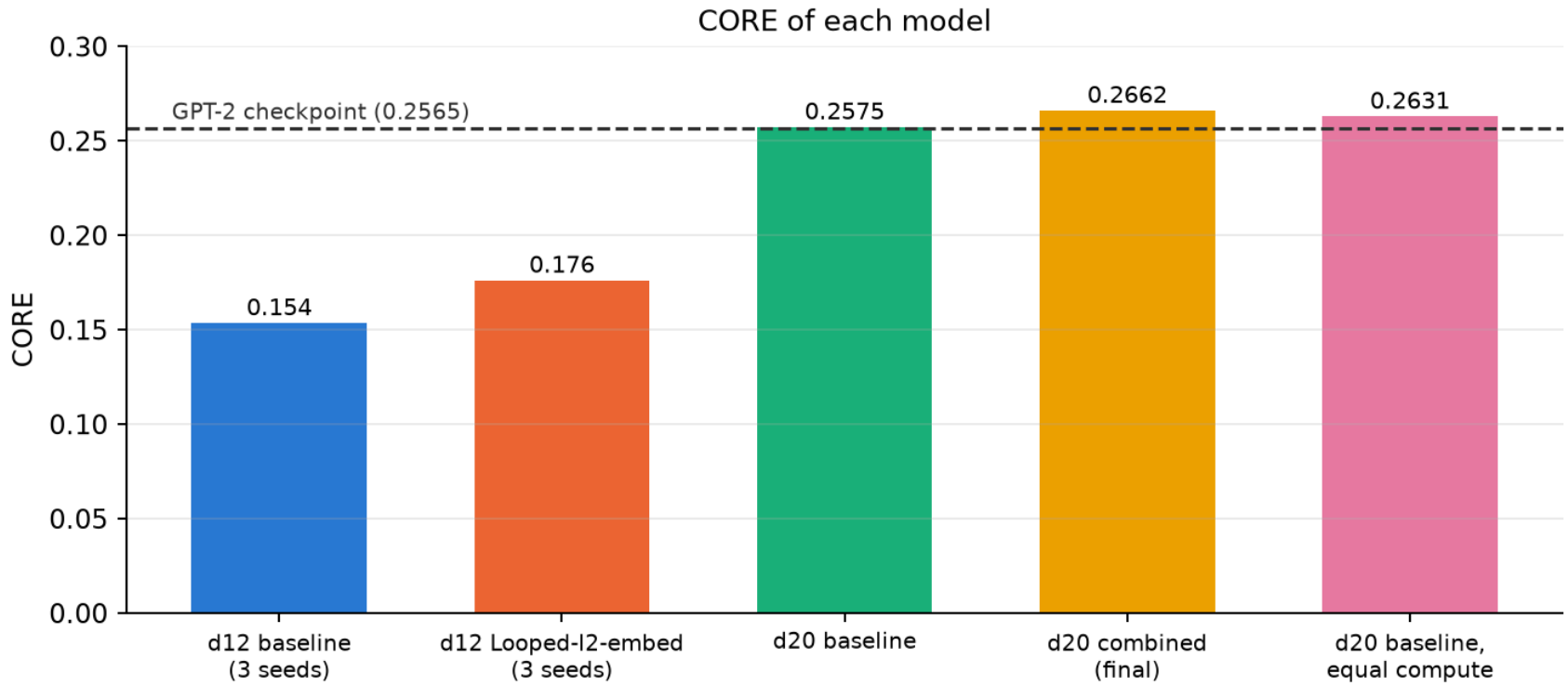


Figure 5.2. CORE of each model against the GPT-2 checkpoint's 0.2565, the reference on nanochat's leaderboard.

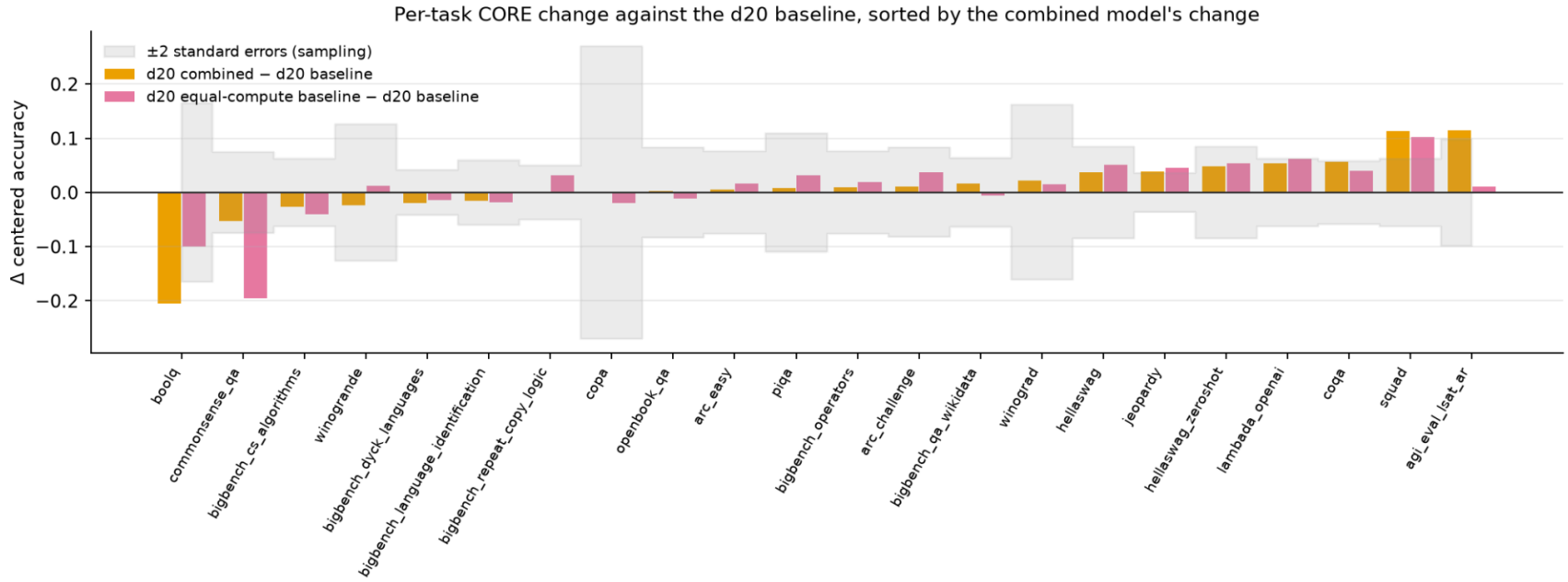


Figure 5.3. Change in each task's centered score against the d20 baseline, for the final model and for the equal-compute baseline. The grey band is ±2 standard errors from sampling the task's examples; it does not include run-to-run noise. Full values are in Table 5.7.

5.4 Scaling law: predicted against actual

Before the d20 run, we predicted its validation bpb from the d12 baseline with a size-only power law:

$$\text{bpb}(d20) = \text{bpb}(d12) \times (N_{20} / N_{12})^{-\alpha}$$

Here N is the number of scaling parameters (435.2M for d20 and 110.1M for d12, a ratio of 3.95), the evaluation set is the held-out validation shard, and α = 0.076 is Kaplan et al.'s (2020) exponent for loss against parameters. This prediction was written in our plan before d20 ran. The compute-law check in Table 5.4 was added afterwards.

Quantity	**Value**
d12 baseline val bpb (mean of 3 seeds)	0.8476
Predicted d20 val bpb, size only, α = 0.076 (before the run)	0.764

Predicted d20 val bpb, Kaplan compute law with exponent 0.050–0.057 and 15.0× the FLOPs (after the run)	0.726–0.740
Actual d20 baseline val bpb	**0.7359**
Implied size exponent α	0.103
Implied compute exponent	0.052

Table 5.4. Predicted and actual d20 baseline. Both sizes use the same architecture, data and recipe.

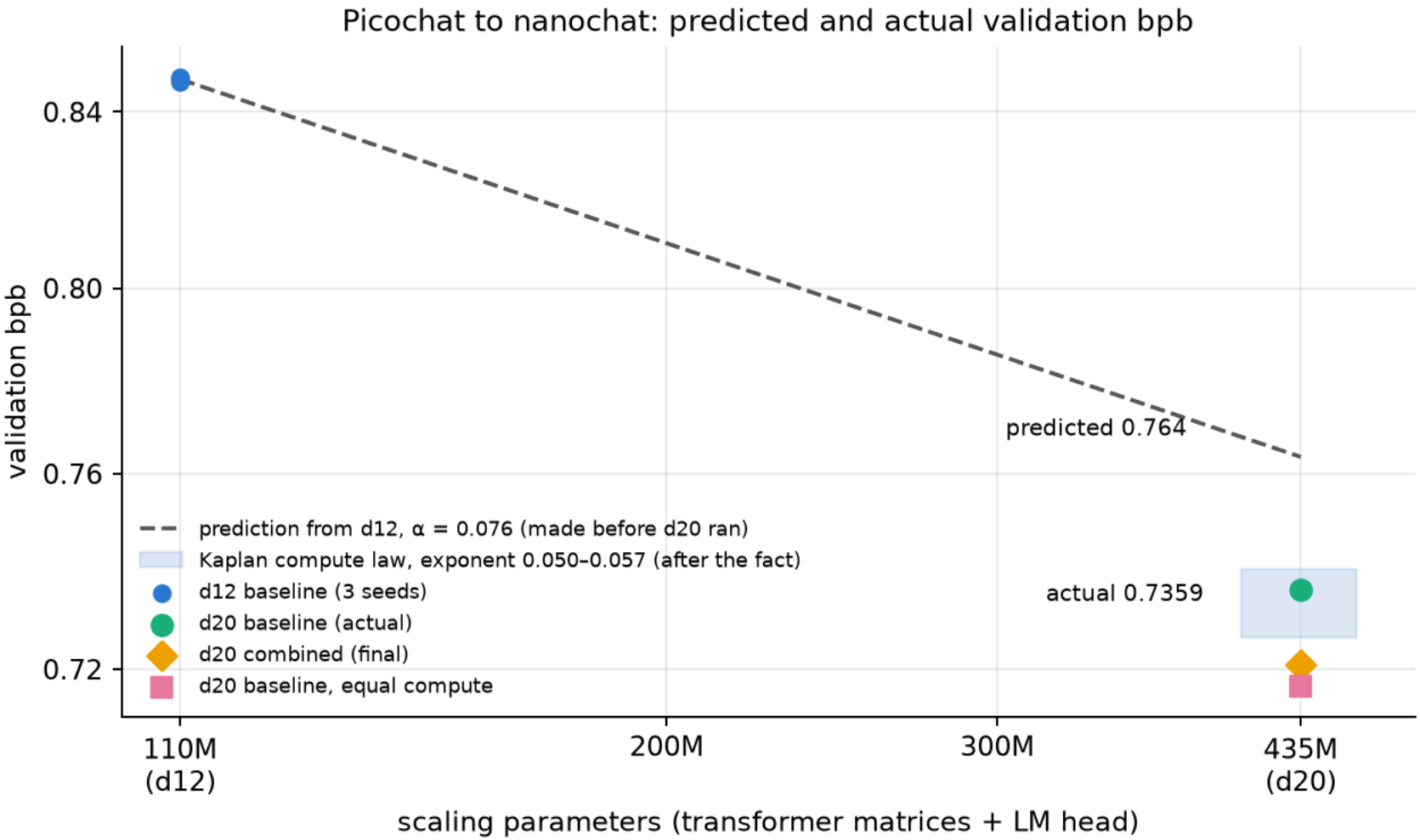


Figure 5.4. Picochat to nanochat. The dashed line is the size-only prediction from the d12 baseline; the blue band is Kaplan's compute law, worked out after the run. The two other d20 models are shown for context.

- **The prediction was too pessimistic.** The d20 baseline beat it by 0.028 bpb, which implies α = 0.103 instead of 0.076.
- **Why α came out larger.** Kaplan et al. measured 0.076 by growing the model while keeping data plentiful, and their exponent for data alone is 0.095. We grew parameters and tokens together, both by 3.95×, so our α is a joint slope for size and data and should be larger than either.

- **The compute law fits.** Kaplan et al.'s exponent for compute is 0.050–0.057. Our total training FLOPs grew 15.0×, which predicts 0.726–0.740, and the actual 0.7359 falls inside (implied exponent 0.052). This treats bpb as loss per token rescaled by a fixed tokenizer and uses nanochat's FLOP estimate instead of Kaplan's 6N per token.
- **The changes lower the curve without steepening it, as far as we can tell.** Part 2 has no d12 run with both changes, so we cannot compute a separate α for the final model. Its d20 lead over the baseline (0.0149) is about 90% of the summed d12 gains (0.0167), and that lead held steady from the first billion tokens to the end (Figure 5.1b).
- **This is a weak test.** With two sizes we can only compute an implied exponent, not fit or test a curve, and each d20 result is a single run.
- **Evaluation setup matters slightly.** The d20 numbers come from training's validation pass on 8 GPUs, while the d12 runs used one GPU. Re-evaluating the final model on one GPU over 21M validation tokens gave 0.7236 instead of 0.7210, with CORE unchanged. We have no one-GPU evaluation of the d20 baseline; if the same offset applied, its implied α would be about 0.100. Comparisons between the d20 runs are unaffected, because they all used the same evaluation pass.

5.5 Commentary

Size and data did most of the work, and the extra tokens mattered.

- The d20 baseline improved on d12 by 0.1117 bpb and 0.104 CORE. After 1.31B tokens, the length of a whole d12 run, it was only level with d12's final value (0.846 against 0.8476), although its learning rate had not yet decayed at that point.
- Most of its lead therefore came from training on four times as many tokens, which fits the joint size-and-data slope in Section 5.4.

Part 2's ablations carried over to d20.

- The final model beat the d20 baseline by 0.0149 bpb, about 90% of what the two d12 gains add up to (0.0167).
- Figure 5.1b shows the lead opening early, reaching 0.015 after about one billion tokens and staying near that value to the end. So the gain is not a late-training artefact.

The gain was bought with compute.

- The final model's training loop cost $84, against $37 for the baseline.
- The baseline trained for the same total FLOPs reached 0.7166 bpb, 0.0043 better than the final model, even though it read the same 110 shards about 2.1 times and saw no new data. Its CORE (0.2631 against 0.2662) is level within noise. This is one run of each.
- Part 2 found the same pattern at d12. Baseline-long beat Looped-l2-embed by 0.0095 bpb at equal compute, and Diff V2's gain matched what its extra 19% compute would buy the baseline (about 0.8414 against 0.8415).
- Of the two changes, Diff V2 was the cheaper. Per unit of extra compute at d12 it gained about 2.3 times as much bpb as looping (0.035 against 0.015). If we had to keep only one change, it would be Diff V2.

CORE passes GPT-2, but the changes cannot take the credit.

- All three d20 models beat the GPT-2 checkpoint's CORE of 0.2565, the reference on nanochat's leaderboard, whose ClimbMix entries score 0.257 to 0.269. The d20 baseline only just clears it (0.2575).
- The jump from d12 to d20 is far larger than seed noise. The final model's 0.0087 lead over the d20 baseline is not, and one run of each cannot show a real CORE difference.

Only a few tasks moved clearly (Figure 5.3, Table 5.7).

- Four tasks moved by more than two standard errors against the d20 baseline: squad (+0.114), agi_eval_lsat_ar (+0.114), jeopardy (+0.038, only just) and boolq (−0.205).
- Squad is the interesting case. At d12 it rose with each change (Diff V2 +0.034, Looped-l2-embed +0.052) but not with extra training (baseline-long 0.180 against 0.175), which fits Part 1's expectation that in-context retrieval benefits most from these changes. At d20, however, the equal-compute baseline gained almost as much (+0.102), so we cannot credit the changes for the d20 gain.
- agi_eval_lsat_ar has few examples, fell from d12 to the d20 baseline, and did not improve with more training, so we read its jump as noise.
- No model has learned boolq. Its centered score is measured against the 62% majority answer, and every model's accuracy is below that: about 53% for the d12 baseline, 59.4% for the d20 baseline, 55.6% for the equal-compute baseline and 51.6% for the final model. Without boolq, the final model's CORE lead over the d20 baseline would be 0.019 instead of 0.0087. We did not check whether its answers shifted toward one class.

5.6 Emergent abilities

Wei et al. (2022) call an ability emergent when smaller models lack it and larger models have it. To look for this, we asked the final d20 model and two d12 models from Part 2 (the seed-42 baseline and Looped-l2-embed) the same 50 factual questions. Each prompt was “Question: …” followed by “Answer:”, decoded greedily for 30 new tokens. An answer counted only if an accepted keyword appeared on its first generated line. The questions and keywords were fixed before any model ran.

The d20 model answered 43 of 50 correctly, against 23 for the d12 baseline and 21 for d12 Looped-l2-embed. On 17 questions the d20 model was right and both d12 models were wrong, and there was no question where a d12 model was right and the d20 model wrong. Table 5.5 lists the first 10 of those 17 in the order they were asked, so we did not pick them by hand. Answers are cut to their first sentence. The other seven cover The Great Gatsby, 1984, The Starry Night, the language of Brazil, the number of continents, the freezing point of water in Fahrenheit and the opposite of hot.

Question	Nanochat (d20 combined)	Picochat (d12 baseline)	Picochat (d12 Looped-l2-embed)

What is the capital of Australia?	**Canberra**	The capital of Australia is Sydney.	The capital of Australia is Sydney, which is located in the Northern Territory.
What is the capital of Japan?	**Tokyo**	The capital of Japan is Kobe, which is the capital of the United States of America.	The capital of Japan is Honshu.
Which country has the largest population in South America?	**Brazil**	South America	South America
Which planet is known as the Red Planet?	**Mars**	The Red Planet is known as the Red Planet.	The planet Mercury is known as the Red Planet.
How many bones are in the adult human body?	**The human body has 206 bones.**	20	The human body is made up of 200 bones.
Who was the first person to walk on the Moon?	**Neil Armstrong**	The first person to walk on the Moon was the American astronaut John F. Kennedy.	The first person to walk on the Moon was Sir John F. Kennedy, who was born on 29 April 1…
Which empire built the Colosseum?	**Rome**	The Colosseum was built by the British Empire in 1789.	The Colosseum was built by the Colosseum of Colosseum in 1571.
Who wrote the Declaration of Independence?	**Thomas Jefferson**	The Declaration of Independence was written by the British government in 1776.	The Declaration of Independence was written by the American Revolutionary War (ARW) part…
Who wrote Romeo and Juliet?	**William Shakespeare**	The first two are the most important of the two.	The Roman Republic was a country in the Eastern Mediterranean, which was divided into tw…
Who wrote Pride and Prejudice?	**Jane Austen**	The first Pride and Prejudice was published in 1913 by the American poet, poet, and play…	The first book of the Pride and Prejudice series was written by the author of the book…

Table 5.5. Ten questions the nanochat answers and both picochats get wrong (the first 10 such questions in the order asked).

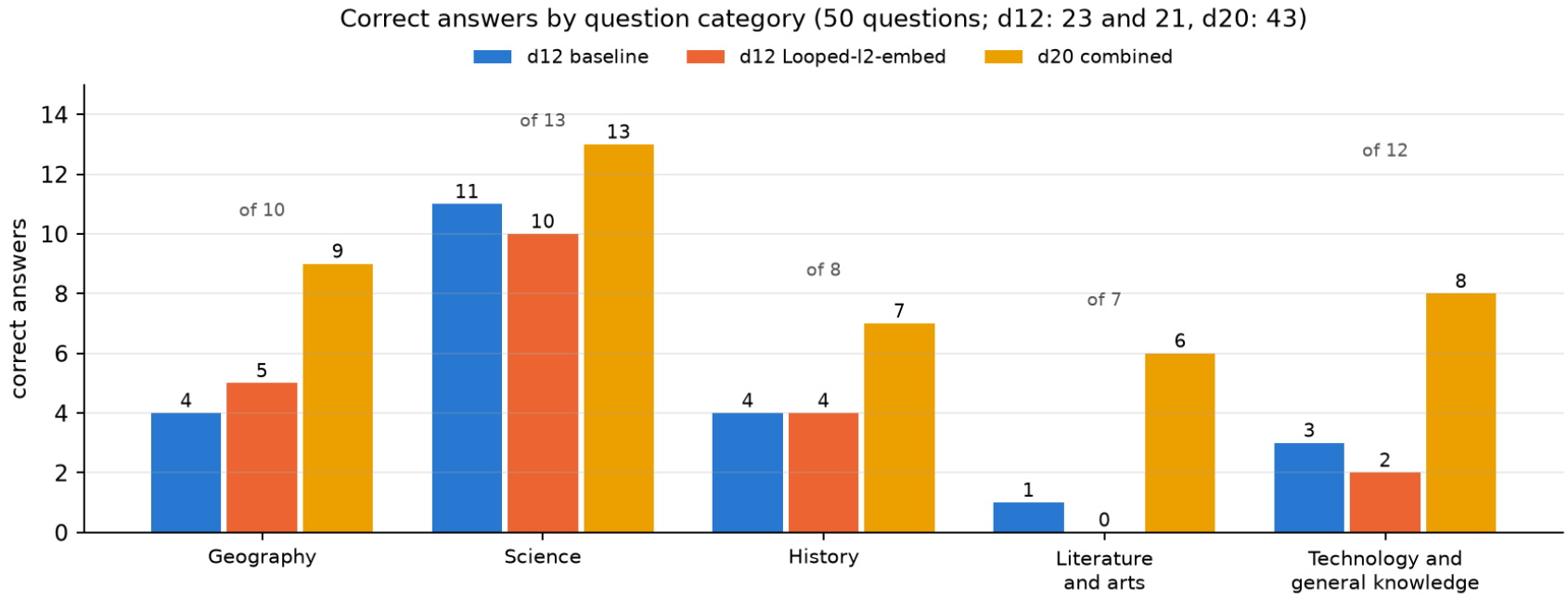


Figure 5.5. Correct answers by question category. The categories were set together with the question list.

What changed between the two sizes:

- **Recalling names.** Literature and arts went from 1 and 0 of 7 at d12 to 6 of 7 at d20. The d12 models recognise the form of "Who wrote X?" and then invent a name or loop. Science was already 11 and 10 of 13 at d12.
- **Answering in the right form.** 28 of the d20 model's 50 answers are four words or fewer, against 14 and 12 at d12. None of its answers repeats itself within the 30 tokens, against 8 and 7 of the d12 answers.
- **What did not emerge: arithmetic and counting days.** All three models miss the square root of 144, 7 times 8 and the number of days in a leap year. The d20 model also misses the capital of Brazil, Britain's wartime prime minister, the composer of the Ninth Symphony and Microsoft's co-founder.

Three limits on these results:

- **The metric may exaggerate the jump.** Schaeffer et al. (2023) argue that apparent emergence can come from all-or-nothing metrics, and ours is one: an answer counts only if the keyword appears. Validation bpb fell only 15% from d12 to d20 while the hit rate nearly doubled, so some of these may be gradual improvements crossing a threshold. Two sizes cannot tell the two apart.

- **The two sizes differ in more than size.** No d12 model with both changes exists, so the gap includes the d20 model's extra parameters, four times the tokens and the architecture changes. We also did not ask the d20 baseline these questions, so we cannot say how much of the gap the changes account for.
- **These are base models that continue text, with no chat training.** The d20 answers are not always clean: its bones answer goes on to invent a breakdown of the skeleton, and its Brazil answer adds that English is also spoken in some areas.

5.7 Cost and hardware

Run	Training time	Container time	MFU	Tokens/s	Peak memory per GPU	Training loop at list price	Billed cost
d20 combined (final)	160.1 min	184.3 min (including the preempted attempt)	42.4%	543K	44.5 GB	$84	$101.61
d20 baseline	69.9 min	81.5 min	45.6%	1.25M	40.9 GB	$37	$44.61
d20 baseline, equal compute	148.1 min	164.9 min	45.9%	1.26M	40.9 GB	$78	$90.64
Total						**$199**	**$236.86**

Table 5.6. Training metrics and cost of the three d20 runs, each on 8×H100. List price is $3.95 per GPU-hour; billed costs include start-up, compilation and evaluation.

- **The cost estimate held.** We projected about 3 hours and $110; training took 2.7 hours and the full bill for the final model was $101.61. That includes about $12 lost to a preemption that hit before the first checkpoint was saved, so training restarted from step 0.
- **Scaling to 8 GPUs worked.** The final run reached 543K tokens/s at 42.4% MFU, about twice the 265K of our 4×H100 test, so doubling the GPUs nearly doubled throughput.
- **The baseline's speed matches its compute.** It ran 2.3 times faster than the final model: 2.14 times from its lower FLOPs per token and the rest from its higher MFU (45.6% against 42.4%).

5.8 Tracking and reproducibility

- **Weights & Biases:** project a3-final holds the d20 combined (fkzk9gfk, tags part5, final), d20 baseline (64i0aafa, trained by Carmen; tags part5, baseline) and the Part 2 d12 runs (group Part 2).

- **Equal-compute d20 baseline:** project vocalocate-a3, run edzeutsd (tags part5, equal-compute), launched as d20-baseline-long. Its per-task CORE is in the run's log.
- **Comparability:** all three d20 runs use the same 32,768-token vocabulary, learning-rate settings and data order. The baseline and final model have the same token budget and finish at the same position in the data (shard 109, row group 24).
- **Code, in the team repository under a3/modal:** `train_d12.py` launches training, `emergent.py` holds the 50 questions, keywords and grader, and `emergent_results.json` holds every answer. The figures in this part were regenerated from the W&B data above.

Final model command: `torchrun --nproc_per_node=8 -m scripts.base_train -- --depth=20 --num-iterations=9960 --total-batch-size=524288 --device-batch-size=8 --max-seq-len=2048 --experiment=combined --diff-v2 --n-loops=2 --n-prelude=1 --n-coda=1 --loop-embed --eval-every=500 --save-every=500 --core-metric-every=999999 --sample-every=-1 --run=d20-combined --model-tag=d20-combined`

5.9 CORE by task

Task	d12 baseline (3 seeds)	d20 baseline	d20 combined	d20 baseline, equal compute	Combined − baseline
arc_easy	0.428	0.616	0.621	0.632	+0.005
piqa	0.375	0.496	0.504	0.528	+0.008
bigbench_qa_wikidata	0.273	0.484	0.500	0.478	+0.016
squad	0.175	0.364	0.478	0.466	+0.114
lambada_openai	0.309	0.420	0.474	0.482	+0.054
bigbench_cs_algorithms	0.449	0.456	0.430	0.416	−0.026
hellaswag	0.157	0.387	0.424	0.437	+0.037
hellaswag_zeroshot	0.156	0.376	0.424	0.429	+0.048
winograd	0.175	0.326	0.348	0.341	+0.022
coqa	0.169	0.268	0.324	0.308	+0.056

copa	0.120	0.300	0.300	0.280	+0.000
openbook_qa	0.097	0.203	0.205	0.192	+0.003
arc_challenge	0.047	0.187	0.197	0.224	+0.011
bigbench_operators	0.117	0.181	0.190	0.200	+0.010
bigbench_language_identification	0.206	0.179	0.164	0.162	−0.015
commonsense_qa	0.175	0.188	0.135	−0.007	−0.052
bigbench_dyck_languages	0.079	0.134	0.114	0.120	−0.020
jeopardy	0.006	0.068	0.106	0.114	+0.038
agi_eval_lsat_ar	0.034	−0.011	0.103	−0.000	+0.114
winogrande	0.060	0.112	0.088	0.124	−0.024
bigbench_repeat_copy_logic	0.000	0.000	0.000	0.031	+0.000
boolq	−0.226	−0.068	−0.274	−0.168	−0.205
Overall CORE	**0.154**	**0.2575**	**0.2662**	**0.2631**	**+0.0087**

Table 5.7. CORE by task, sorted by the final model's centered score. Centered scores correct for chance on multiple-choice tasks, and CORE is their mean.

Works Cited

Allen-Zhu, Z. (2025). Physics of language models: Part 4.1, architecture design and the magic of canon layers. *Advances in Neural Information Processing Systems, 38*, 47319–47339. https://doi.org/10.52202/085713-1410

Bae, S., Fisch, A., Harutyunyan, H., Ji, Z., Kim, S., & Schuster, T. (2025). *Relaxed recursive transformers: Effective parameter sharing with layer-wise LoRA*. International Conference on Learning Representations (ICLR). https://arxiv.org/abs/2410.20672

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). *BERT: Pre-training of deep bidirectional transformers for language understanding* [Preprint]. arXiv. https://arxiv.org/abs/1810.04805

Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., & Kaiser, Ł. (2019). *Universal transformers*. International Conference on Learning Representations (ICLR). https://arxiv.org/abs/1807.03819

Geiping, J., McLeish, S., Jain, N., Kirchenbauer, J., Singh, S., Bartoldson, B. R., Kailkhura, B., Bhatele, A., & Goldstein, T. (2025). *Scaling up test-time compute with latent reasoning: A recurrent depth approach* [Preprint]. arXiv. https://arxiv.org/abs/2502.05171

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., . . . Sifre, L. (2022). *Training compute-optimal large language models* [Preprint]. arXiv. https://arxiv.org/abs/2203.15556

Hsieh, C.-P., Sun, S., Kriman, S., Acharya, S., Rekesh, D., Jia, F., Zhang, Y., & Ginsburg, B. (2024). *RULER: What's the real context size of your long-context language models?* In *Proceedings of the Conference on Language Modeling (COLM)*. https://arxiv.org/abs/2404.06654

Jin, H., Han, X., Yang, J., Jiang, Z., Chang, C.-Y., & Hu, X. (2023). *GrowLength: Accelerating LLMs pretraining by progressively growing training length* [Preprint]. arXiv. https://arxiv.org/abs/2310.00576

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). *Scaling laws for neural language models* [Preprint]. arXiv. https://arxiv.org/abs/2001.08361

Karpathy, A. (2025). *nanochat* [Computer software]. GitHub. https://github.com/karpathy/nanochat

Karpathy, A. (2026). *Experiment log* [Repository file]. GitHub. https://github.com/karpathy/nanochat/blob/92d63d4e8bb4df75c3b71618f31ddde2378b2bcd/dev/LOG.md

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics, 12*, 157–173. https://doi.org/10.1162/tacl_a_00638

Miller, E. (2024). *Adding error bars to evals: A statistical approach to language model evaluations* [Preprint]. arXiv. https://arxiv.org/abs/2411.00640

Press, O., Smith, N. A., & Lewis, M. (2021). Shortformer: Better language modeling using shorter inputs. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)* (pp. 5493–5505). Association for Computational Linguistics. https://doi.org/10.18653/v1/2021.acl-long.427

Saunshi, N., Dikkala, N., Li, Z., Kumar, S., & Reddi, S. J. (2025). *Reasoning with latent thoughts: On the power of looped transformers*. International Conference on Learning Representations (ICLR). https://arxiv.org/abs/2502.17416

Schaeffer, R., Miranda, B., & Koyejo, S. (2023). Are emergent abilities of large language models a mirage? *Advances in Neural Information Processing Systems, 36*, 55565–55581. https://doi.org/10.52202/075280-2425

University of Toronto. (n.d.). *Class slides* [Google Slides presentation]. https://docs.google.com/presentation/d/1uy_fg1JZRgGVRFCtpqokaceN5aP7Yc6SGJEAt-GtOI4/edit

Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., Chi, E. H., Hashimoto, T., Vinyals, O., Liang, P., Dean, J., & Fedus, W. (2022). Emergent abilities of large language models. *Transactions on Machine Learning Research*. https://arxiv.org/abs/2206.07682

Ye, T., Dong, L., Xia, Y., Sun, Y., Zhu, Y., Huang, G., & Wei, F. (2025). *Differential transformer*. International Conference on Learning Representations (ICLR). https://arxiv.org/abs/2410.05258

Ye, T., Dong, L., Sun, Y., & Wei, F. (2026, January 20). *Differential transformer V2*. Hugging Face. https://huggingface.co/blog/microsoft/diff-attn-v2

Zhu, R.-J., Wang, Z., Hua, K., Zhang, T., Li, Z., Que, H., Wei, B., Wen, Z., Yin, F., Xing, H., Li, L., Shi, J., Ma, K., Li, S., Kergan, T., Smith, A., Qu, X., Hui, M., Wu, B., . . . Eshraghian, J. (2025). *Scaling latent reasoning via looped language models* [Preprint]. arXiv. https://arxiv.org/abs/2510.25741